

华中科技大学

HUAZHONG UNIVERSITY OF SCIENCE AND TECHNOLOGY



面向 AI 智能体的操作系统内核 Agent-OS

队伍名称	AAAos 翻新队
赛 题	Proj61
指导老师	谭志虎
项目成员	谭雨馨 何思珍 万翀宇

二〇二六年 八月

目录

目录II

1 项目简述1

 1.1 项目背景1

 1.2 项目创新点2

2 项目整体设计2

 2.1 系统框架2

 2.2 模块简介3

 2.3 任务完成情况6

 2.4 创新点7

 2.4.1 创新一：跨 Agent 查询结果共享与状态一致性7

 2.4.2 创新二：事件感知的 Agent Loop 与动态调度8

 2.4.3 创新三：动态工具注册（LLM Skill 可插拔工具模型）9

 2.4.4 创新四：内核原生 Workflow 多 Agent 协作治理11

 2.4.5 创新五：多 Agent 安全管控体系12

3 详细设计与实现13

 3.1 Agent 进程结构与生命周期设计13

 3.1.1 关键结构13

 3.1.2 关键函数13

 3.1.3 workflow 与多 agent 协作14

 3.2 Context 上下文系统16

 3.2.1 关键结构16

 3.2.2 Path 结构17

 3.2.3 核心函数18

 3.2.4 淘汰算法-FIFO19

 3.2.5 摘要校验与一致性设计19

3.3 Tool Call 与进程互动	21
3.3.1 内建工具	21
3.3.2 动态工具注册与调用	23
3.4 AgentFS 文件查询与协作	26
3.4.1 AgentFS 结构扩展	26
3.4.2 索引设计	26
3.4.3 文件查询机制	27
3.4.4 共享缓存机制	28
3.4.5 权限管控	28
3.4.6 文件协作与租约机制	29
3.4.7 文件变更事件通知机制	30
3.5 Agent Loop、心跳与事件感知调度	31
3.5.1 心跳机制	31
3.5.2 时间轮设计	32
3.5.3 统一事件置位逻辑	33
3.5.4 软预算机制	34
3.6 安全控制与多 Agent 协作治理	35
3.6.1 角色与 capability 模型	35
3.6.2 工具权限控制	36
3.6.3 Context Guard Page 与校验设计	36
3.6.4 Workflow 与级联终止治理	37
4 CodeLab 综合场景	38
4.1 场景架构	38
4.2 对比测试	38
4.3 测试结果	40
5 实际 LLM 测试	41
6 性能测试	43
6.1 文件查询扫描量对比	43
6.2 共享缓存复用效果	44
6.3 AgentOS CPU 开销与唤醒延迟	44
6.4 调度机制公平性	45

7 AI 使用情况	48
-----------------	----

1 项目简述

1.1 项目背景

大语言模型驱动的 AI 智能体正在革新人机交互与程序开发的范式。以 AI 变成工具为例，普遍采用 “Agent 主动发起查询→系统执行返回→Agent 基于结果再决策” 的循环工作范式。该范式依赖结构化工具交互，且需要持续留存多轮执行上下文以支撑后续推理。但当前智能体全部部署于应用层，操作请求需逐层穿越 Shell、运行时库等软件栈抵达底层内核。系统无法直接感知 Agent 的身份、意图和协作关系，也难以对高频文件查询、空闲等待和多 Agent 调度进行有针对性的优化。

Agent-OS 在 xv6-riscv 基础上采用在传统进程之上叠加 Agent 语义层的路线：保留进程、页表、文件系统和调度器等基础机制，同时在 PCB、系统调用和内核模块中加入 Agent 身份、Context Path、Tool Router、AgentFS、事件驱动 Agent Loop、事件感知调度、Capability 权限、Lease 文件协作以及 Workflow 治理。

Agent 运行需求	传统 xv6 的典型限制	Agent-OS 对应机制
连续 Think-Act-Observe	无 Agent Loop 与专属状态	Agent Runtime + Agent Loop 状态机
结构化能力调用	能力分散在零散 syscall / 用户态逻辑	Tool Router + Built-in/Dynamic Tool
语义文件检索	主要依赖路径和遍历	AgentFS 属性/摘要索引 + query_file
多 Agent 协作	IPC 不表达 Agent 身份与工作流语义	FIFO Mailbox + Agent Discovery + Workflow
空闲等待	用户态轮询浪费 CPU	agent_wait 阻塞 + Event Wakeup
优先级/事件紧急度	普通轮转不理解事件语义	Event-aware Scheduler
安全边界	工具访问缺少 Agent 角色语义	Role + Capability + Group + Lease

本项目在原 OS 的基础上进行系统化扩展，补充模块内部数据结构、关键函数、生命周期、调用链、设计权衡、实现边界、CodeLab 综合 workflows 以及完整量化实验。

1.2 项目创新点

AgentOS 在 xv6-riscv 基础上分层扩展，整体划分为七大模块：

- Agent 生命周期
- Context 上下文
- 标准化工具调用
- AgentFS 语义文件
- 心跳事件调度
- 多 Agent 安全治理
- CodeLab 综合演示

系统创新设计专属状态机、身份代次与 Workflow 工作机制，依靠身份代次解决 PID 复用识别漏洞，搭配级联终止逻辑，避免协作进程遗留孤儿实例。其次实现内核化 Agent Loop 运行框架，采用时间轮高效管理周期性心跳，统一封装 watch 订阅、agent_wait 阻塞等待机制，各类唤醒源汇聚为统一事件位图，彻底替代用户态忙等轮询，空闲等待阶段 CPU 开销降至零。

文件层创新推出 AgentFS，为文件附加结构化属性与倒排索引，搭配工作组共享查询缓存大幅减少文件扫描，同时引入排他租约机制管控并发修改冲突。工具层面构建统一 RPC 调用体系，区分内核内置工具与 Agent 动态注册服务，基于 Capability 位图实现分层权限管控。上下文模块采用“用户可读文本 + 内核摘要链”双写模型，结合奇偶快照、链式摘要完成三层数据可信校验，完整留存可追溯执行轨迹。

2 项目整体设计

2.1 系统框架

本项目基于 MIT xv6 教学操作系统进行二次开发，整体内核代码划分为原生 xv6 基础模块与 Agent-OS 智能体扩展子系统两大层级，在保留 xv6 原有进程、内存、文件、中断、系统调用等完整基础能力的前提下，针对 AI 智能体运行需求完成模块化扩展，各原生模块改造与新增智能体子系统分工清晰、低耦合集成。

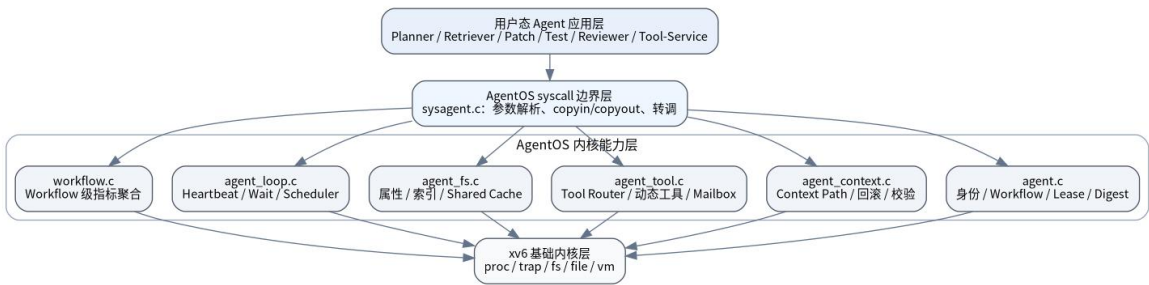


图 2.1 系统框架图

2.2 模块简介

项目由七大模块构成，其中主要的六大功能模块简介如下：

模块	当前实现要点	主要价值
Agent 进程设计	Agent 身份、生命周期、Context、Workflow 元数据	建立 Agent 原生执行实体
Context	用户态实体数据 + 内核元数据、FIFO、guard page、digest chain	高频可读且受内核治理的运行记忆
Tool Call	统一 tool_call，内建工具与动态 Skill 同接口	能力总线与可插拔扩展
AgentFS 文件系统	属性/摘要、倒排/哈希索引、query planner、shared cache、lease	语义查询与文件协作
Agent Loop、Scheduler	FIFO Mailbox、MESSAGE/FILEMOD/HEARTBEAT/timeout、priority、event weight、aging、boost、vruntime、budget、deadline	事件驱动协作、面向 Agent 事件语义的动态调度
Governance 安全管理	Role + Capability、Group、Lease、Workflow、cascade terminate	最小权限和协作治理

- Agent 进程设计

该模块是 AgentOS 整体骨架，在 xv6 原生 PCB 基础上扩展专属语义字段，设计 IDLE、READY、RUNNING、WAITING、ROLLED_BACK、DONE

六态状态机完整管控 Agent 全生命周期。模块引入 identity_generation 身份代次机制，解决 PID 复用引发的所有权误判问题；配套 Workflow 工作流体系，划分 Planner 主控与 Worker 工作角色，支持工作流创建、成员加入退出、级联终止逻辑，Leader 退出时内核自动清理同组所有 Agent，杜绝孤儿进程。同时规范 fork、exit 生命周期行为，fork 时仅继承基础身份，重置权限、心跳与上下文，运行期仅支持降权不可私自提权；退出时自动清理时间轮注册、文件租约、动态工具与消息队列，释放全部内核资源。

● Context 上下文子系统

Context 是内核托管的 Agent 专属执行记忆区，采用用户可读文本区 + 内核摘要环双写模型，同步留存工具调用记录与可信校验指纹。内存布局划分固定 Header、变长 Path 记录区与防护 Guard 页，Guard 页拦截越界访问；依靠 context_generation 奇偶代数区分写入中、稳定两种快照状态，避免用户读取半写脏数据。系统采用 FIFO 容量淘汰策略，超出节点上限自动丢弃最早记录；配套链式摘要校验机制，每条记录哈希关联前一条指纹，可检测篡改、断链、半写入故障。同时提供查询、回滚、清空、校验四类系统调用，支持 Agent 校验文件版本，自动识别过期上下文。整套机制兼顾可读性与数据可信，为 Agent 推理、行为审计、故障回溯提供可验证执行轨迹。

● Tool Call 统一工具调用模块

该模块构建标准化工具交互层，统一封装文件读写、消息通信、系统查询、动态服务等全部能力，分为内核内置工具与 Agent 动态注册工具两类，采用统一键值对请求 - 响应格式。内置工具通过 builtin_policies 绑定 Capability 权限，调用前完成位图校验；动态工具基于内核请求槽实现轻量 RPC，Agent 可自主注册服务，通过组可见性控制访问范围，服务退出时自动清空未完成请求，避免悬挂调用。内置消息邮箱配套通信逻辑，采用有界 FIFO 队列，消息到达自动触发 MESSAGE 事件，拥塞时返回 BUSY 背压，保证消息有序无重复。配套 Agent 发现工具，可按角色、能力、分组检索协作对象，支撑点对点消息与服务调用两种协作模式。

● AgentFS 语义文件系统

AgentFS 不重构 xv6 底层 inode，通过扩展文件属性、内容摘要、倒排索引实现面向 Agent 语义检索。为每个文件挂载 type、module、tag 结构化属性与内容前缀 summary，基于属性构建 Posting 倒排哈希索引，查询时仅匹配少量候选文件，摒弃全目录遍历；同工作组共享查询缓存，大幅削减重复扫描开销。引入 inode 级排他租约机制，修改文件前申请独占租约，超时或进程退出自动回收，规避并发写冲突。配套分层文件通知：全局广播所有 FILEMOD 订阅 Agent，精准监听仅唤醒绑定特定 inode 的 Review 类 Agent，实现变更事

件驱动。Agent 可按业务语义检索目标文件，配合事件通知形成检索 - 修改 - 审核自动化协作链路，显著降低文件 IO 与系统调用开销。

● Agent Loop、心跳与事件调度模块

本模块实现内核化 Agent 循环，依托时间轮、agent_wait 构建完整事件驱动调度闭环。时间轮将 Agent 心跳 deadline 映射至循环桶数组，时钟中断仅扫描当前桶，消除全进程无效遍历；agent_tick 周期驱动心跳、租约清理、预算补充。设计 watch 订阅接口，支持消息、文件变更事件订阅，所有唤醒源统一聚合至 pending_events 位图，遵循治理事件 > 消息 > 文件变更 > 心跳固定优先级。支持持续循环或终止两种模式，无唤醒源、超时两种保护逻辑，空闲 Agent 完全阻塞不占用 CPU。调度层新增事件感知打分函数，结合优先级、软预算惩罚、事件权重分配 CPU 时间，预算周期性自动补充，兼顾资源约束与低优先级进程防饥饿。整套机制替代用户态轮询，实现周期自检、消息协作、文件感知三类场景统一调度，大幅降低空闲阶段 CPU 损耗。

● 安全与多 Agent 协作治理模块

模块以“角色 + Capability 位图”构建分层权限体系，运行仅允许削减权限，禁止私自提升。从多维度搭建防护体系：文件修改依靠 inode 租约隔离并发冲突；Context 依托防护页、奇偶快照、链式摘要三层防篡改；Workflow 提供级联终止，防止失控 Worker 残留；动态工具、查询缓存通过 agent_group 划分协作边界，跨组访问需标记公开。系统内置完整审计与追踪接口，权限拒绝、工具调用、进程终止均生成审计日志，记录发起身份代次避免 PID 混淆。同时规范 fork 继承策略，子进程仅复制基础 Agent 身份，重置角色与权限，防止权限泄露。覆盖文件、工具、上下文、工作流全场景管控，兼顾协作灵活性与操作可控性，所有关键行为可追溯、越权操作可拦截，为多 Agent 并行运行提供轻量化安全隔离框架。

● CodeLab 综合演示与 LLM 桥接模块

CodeLab 是验证全内核机制的端到端多 Agent 工作流，搭建 Planner、检索、修复、测试、审核、工具服务六类 Agent 协作完成代码缺陷修复，串联 AgentFS 索引缓存、文件租约、FILEMOD 事件、动态工具、Workflow 级联、Context 校验全部核心能力。模块实现 LLM 策略层与 AgentOS 执行层分离，通过串口文本协议完成宿主机大模型与内核通信，LLM 仅下发高层决策，文件修改、工具调用、多进程协作全部在内核权限约束下执行，隔离大模型直接操作底层资源风险。内置 Workflow 量化统计组件，汇总工具调用、文件扫描、等待时长等全链路指标；配套双组对照实验，在业务结果完全等价前提下，对比原生 xv6 与 AgentOS 资源消耗，量化验证索引、缓存、事件等待带来的扫描、读取、轮询降幅。

2.3 任务完成情况

本项目已完成赛题要求的基础任务，并在此基础上围绕 Agent 协作场景实现了若干具有代表性的系统创新。整体上，系统已经在 xv6-riscv 上实现了 Agent 进程抽象、结构化工具调用、上下文路径管理、AgentFS 文件查询扩展、Agent Loop 事件等待与调度机制，以及综合演示场景 CodeLab，能够在 QEMU 中稳定运行并完成多 Agent 协作任务。

● 任务一 Agent 进程创建与地址空间设计

项目不仅扩展了 PCB 中的 Agent 身份、事件、调度和工作流字段，还为 Agent 分配了独立的用户态 Context 区，并通过 guard page 提升上下文区域的安全性。这使 Agent 不再是普通进程的简单复用，而成为具备内核可识别身份和独立运行语义的执行实体。

● 任务二 结构化交互接口

系统以统一 Tool Call 为核心，构建了带权限策略的内核能力目录。相比传统系统调用接口，本项目的创新在于：工具调用统一经过权限校验、生命周期状态切换与 Context 自动记录，使工具执行天然带有可治理、可追踪和可审计特征。同时，项目进一步扩展了动态工具注册机制，使用户态服务进程能够按需注册新工具，从而形成接近 LLM Skill 模式的可插拔工具架构。

● 任务三 上下文路径管理

系统不仅实现了 push/query/rollback/clear 等基本操作，还通过 context_generation 奇偶快照协议和 context_digests[] 链式摘要，实现了对上下文历史的一致性保护。进一步地，项目引入了版本感知 Context 机制，能够在文件修改后识别历史上下文已变为 STALE，体现了上下文管理从“可存储”向“可验证、可判定有效性”的扩展。

● 任务四 文件系统查询扩展

项目的核心创新在于将 AgentFS 构建为面向语义查询的文件系统机制。系统通过 inode 属性、摘要索引和 Posting 结构，使文件查询从路径遍历转向属性筛选；同时，引入跨 Agent 共享查询缓存，将多 Agent 对相同查询结果的复用下沉为内核能力，并以文件系统全局版本号驱动缓存自动失效，从而保证查询结果共享与状态一致性。

● 任务五 Agent Loop 内核运行机制

项目实现了消息、文件修改和心跳三类事件的统一等待/唤醒模型，并将事件权重正式纳入调度打分逻辑。其创新点在于，调度器不再只依赖静态优先级，而能够根据消息、文件变化等事件语义动态倾斜资源。决赛阶段进一步引

入了时间轮心跳管理，避免每个 tick 全量扫描全部 Agent，显著增强了周期性唤醒路径的可扩展性。

● 任务六 综合场景 Codelab

项目实现了 CodeLab 多 Agent 代码修复场景，并进一步完成了内核原生 Workflow 治理。系统通过 workflow_id、Leader 代次信息和 agent_cascade_terminate()，将多 Agent 协作从松散消息关系提升为内核可识别、可终止、可统计的任务组织结构；同时实现了 Workflow 级指标汇总，使内核首次能够以“任务”而非“单进程”为单位观察 Agent 工作负载。

2.4 创新点

2.4.1 创新一：跨 Agent 查询结果共享与状态一致性

传统多 Agent 系统中，相同文件查询通常由各 Agent 在用户态分别执行和缓存，既会重复扫描文件元数据，又难以保证缓存结果与文件状态的一致性。针对这一问题，AgentOS 将查询结果复用下沉为内核原生能力，在内核中统一维护跨 Agent 共享查询缓存，并进一步引入文件系统全局版本与版本感知 Context 机制，使查询结果复用和上下文有效性判断同时具备可验证的一致性语义。

首先，内核维护 shared_query_cache 全局缓存表，以完整查询参数字符串作为唯一缓存键，缓存结构化查询结果，并绑定 owner_pid、owner_group、refcnt 以及生成时的文件系统全局版本号。系统通过 shared_query_cache_lookup() 与 shared_query_cache_store() 两个核心函数完成缓存命中判断、结果写回与引用统计管理。与普通用户态局部缓存相比，该机制能够在多个 Agent 并发执行相同检索任务时直接复用已有结果，从而避免重复扫描 AgentFS 元数据，显著降低查询 CPU 消耗。

其次，为保证跨进程共享的安全边界，系统设计了 cache_access_allowed() 与 query_allows_public_share() 两级权限校验逻辑。仅当查询参数显式声明 public=true、查询限定 owner=system，或当前调用 Agent 与缓存归属 Agent 处于同一 agent_group 时，才允许复用缓存结果。由此，查询缓存不再只是性能优化结构，而成为受协作边界约束的系统级共享资源，在提升查询效率的同时避免不同工作流之间无控制的数据扩散。

再次，AgentOS 通过文件系统全局版本实现查询缓存自动失效。每当文件属性新增、删除或修改，系统都会执行 agent_file_version_bump() 递增全局元数据版本，并同步重建文件索引。缓存条目在生成时写入对应版本号，后续查

询若发现缓存版本与当前 `agent_file_version` 不一致，则立即判定缓存失效并重新执行真实查询。这种设计避免了复杂的逐文件依赖跟踪，而通过单调版本号实现了低成本、强一致的全局失效策略。CodeLab 场景已经验证：文件修改后旧查询缓存会自动失效，重复检索将重新走索引与扫描路径。

在此基础上，系统进一步扩展了版本感知 Context 机制。除共享查询缓存外，Context 在执行 `read_file` 等文件读取行为时，还会记录对应 `inode` 对象版本；随后通过 `context_validate` 按需判定该上下文引用当前处于 VALID、STALE 或 DELETED 状态。其含义分别为：文件内容未变化、文件已被修改导致上下文失效、文件对象已不存在。这样，AgentOS 不仅能判断“查询结果是否可复用”，还能判断“先前读取过的上下文是否仍然可信”。CodeLab 场景已经完整演示了这一链路：文件修改后共享缓存失效，旧 Context 从 VALID 变为 STALE，再次读取目标文件后恢复为 VALID。这说明系统首次把查询缓存一致性与上下文版本感知统一纳入同一套内核状态管理框架。

最后，在 `query_file` 的完整执行链路中，系统优先尝试共享缓存命中。若命中成功，则直接返回已有结果，并标记 `cache_hit=1`、`fs_scanned=0`，完全跳过文件索引和元数据扫描；若未命中，则执行完整索引匹配与文件扫描，完成后再将结果写回共享缓存，并附带缓存命中标志、扫描统计与版本信息供实验验证。由此，AgentOS 实现了“共享查询结果复用 + 全局版本失效 + 版本感知 Context 校验”的统一状态一致性机制，这是区别于传统用户态局部缓存方案的重要创新。

2.4.2 创新二：事件感知的 Agent Loop 与动态调度

传统 xv6 调度器仅面向通用进程执行，不理解智能体当前正在等待消息、响应文件变更还是仅执行周期性轮询，因此无法体现任务语义差异。针对这一局限，AgentOS 将 Agent Loop、事件订阅、等待/唤醒和调度打分整合为统一内核机制，构建了一个能够感知事件紧急程度和运行状态的动态调度框架。

首先，系统在 `struct proc` 中扩展了 Agent 专属字段，用于记录 `watch_mask`、`pending_events`、`last_wakeup_reason`、`wait_deadline`、`agent_sched_priority`、`agent_sched_quota`、`agent_sched_budget`、`agent_sched_boost` 等信息，作为调度决策的数据源。与传统 PCB 只保存基础执行状态不同，这些字段使内核能够理解一个 Agent 当前是否携带待处理消息、是否刚被文件修改唤醒、是否即将超时，以及其预算是否充足。

其次，系统将三类核心事件完整接入内核执行链路。心跳事件由 `trap.c` 中的时钟中断周期调用 `agent_tick()` 推动；消息事件由 `send_message` 工具写入目

标 Agent 邮箱后自动触发 AGENT_EVENT_MESSAGE；文件修改事件则通过 agent_notify_file_modified() 与 agent_signal_filemod() 注入 AGENT_EVENT_FILEMOD，既支持精确监听特定 inode，也支持全局文件变化广播。各 Agent 可通过 agent_watch()、agent_unwatch() 和 agent_watch_file() 自主订阅或取消监听，从而形成显式事件驱动的 Agent Loop。

在调度策略上，系统封装 agent_schedule_score() 作为统一打分函数，将静态优先级、aging、待处理事件权重、临时调度增益、vruntime 惩罚、预算惩罚及 deadline bonus 共同纳入调度分值计算。其中，事件权重体现了明确的语义偏好：MESSAGE 权重最高，FILEMOD 次之，HEARTBEAT 最低，从而实现“优先处理协作输入，再处理环境变化，最后处理周期性轮询”的调度顺序。该机制突破了 xv6 原生静态轮转思想，使调度器能够主动向更紧急、更具协作价值的 Agent 倾斜资源。

在决赛增强中，系统进一步引入了时间轮式心跳管理机制。传统做法通常需要在每个 tick 上遍历全部 Agent 检查心跳是否到期，而 AgentOS 使用时间轮将 heartbeat_deadline 映射到固定桶中，只扫描当前 tick 所对应的桶，并在到期后将周期任务重新排回未来桶位。其核心函数包括 agent_heartbeat_wheel_insert_locked()、agent_heartbeat_wheel_remove_locked()、agent_heartbeat_wheel_reschedule() 与 agent_heartbeat_wheel_remove()。这一设计显著减少了无效扫描开销，使系统不再需要在每个时钟周期全量遍历所有 Agent，从而提高了心跳驱动路径的可扩展性。

此外，系统还实现了软预算控制机制。对于配置了 quota 的 Agent，每次运行都会消耗 agent_sched_budget；预算耗尽后并不会被硬性禁止调度，而是通过降低调度得分进行软惩罚，并由 agent_replenish_budgets() 在后续周期统一恢复预算。这种方式在保持高优先级事件响应能力的同时，避免了单个 Agent 长时间独占 CPU，也减少了硬配额策略下“资源用尽即完全停摆”的问题。

因此，AgentOS 的这一创新并不只是“给进程加优先级”，而是构建了一套由事件订阅、阻塞等待、统一唤醒、语义打分和时间轮心跳共同组成的 Agent 原生运行框架，使调度从“按进程轮转”提升为“按事件和任务语义动态驱动”。

2.4.3 创新三：动态工具注册（LLM Skill 可插拔工具模型）

传统操作系统中的工具能力通常由内核硬编码实现，新增能力往往需要修改内核代码并重新编译，无法适应智能体场景下工具按需扩展的要求。针对这

一问题，AgentOS 设计了“内核统一路由 + 用户态服务进程实现”的动态工具模型，使工具能力可以像 LLM Skill 一样按需插拔，同时仍保持统一调用接口和内核安全治理。

系统围绕动态工具构建了三段式交互协议。首先，工具服务进程通过 `agent_tool_register` 向内核注册工具名及相关标识；其次，服务进程通过 `agent_tool_recv` 阻塞读取由内核转发的调用请求；最后，服务进程完成执行后，通过 `agent_tool_reply` 将执行结果和状态码回传内核，并由内核唤醒原调用方 Agent。由此，用户态工具进程被纳入受控的内核调用链，形成了“注册—接收—执行—回复”的标准服务模型。

在统一路由逻辑上，所有 Agent 的工具请求均首先进入 `agent_tool_call()`。系统优先匹配内核内建工具，若匹配失败则进入 `agent_dynamic_tool_call()`，由内核在全局请求槽中记录调用方 PID、服务 PID、参数、结果缓冲等信息，并阻塞调用方等待服务结果返回。因此，对上层 Agent 而言，工具调用接口始终保持统一，调用方无需关心目标工具究竟是内建实现还是用户态服务实现。

动态工具模型的关键创新之一在于其工具生命周期与异常治理能力。第一，服务进程退出时，内核会自动注销该进程名下注册的全部动态工具，避免产生悬空工具项；第二，对于已经投递但尚未完成的请求，若服务进程异常退出，系统会将其状态统一置为 `SERVICE_GONE` 并唤醒调用方，保证调用方不会永久阻塞；第三，当全局请求槽资源耗尽时，系统显式返回忙状态，而非静默失败，从而形成明确的背压语义。也就是说，动态工具不仅支持扩展能力注入，还具备完整的生命周期清理与异常收敛机制。

在权限隔离方面，系统设计了多层安全控制。首先，普通非 Agent 进程不允许注册或调用动态工具；其次，调用前统一执行 `capability` 检查，确保只有具备对应能力的 Agent 才能访问工具；再次，通过 `agent_same_group_or_public()` 实现 `group/public` 边界控制，未标记 `AGENT_TOOL_FLAG_PUBLIC` 的工具仅允许同协作组内 Agent 调用，标记为公开后才允许跨组访问。这样，动态工具既可服务 workflow 内专用协作，又可在必要时作为系统公共能力对外提供。

此外，系统还支持统一工具发现。调用 `tool_list` 时，内核通过 `agent_copy_tool_list()` 将当前进程可见的内建工具与动态工具合并输出，使调用方获得的是统一工具目录，而无需区分工具来源。由此，AgentOS 首次将“可插拔工具 + 生命周期治理 + 权限边界控制 + 统一发现”整合为同一套内核能力体系，为未来面向 LLM Skill 的扩展执行模型奠定了基础。

2.4.4 创新四：内核原生 Workflow 多 Agent 协作治理

传统操作系统仅理解独立进程与父子关系，并不存在“工作流”这一协作抽象。多 Agent 系统若完全依赖用户态消息在运行时临时拼接协作关系，往往会面临主控退出后残留孤儿等待进程、工作流身份无法统一识别、过程指标难以汇总等问题。针对这一局限，AgentOS 在内核层原生引入 Workflow 治理机制，将多 Agent 协作从“松散消息连接”提升为“内核可信管理的任务组织单元”。

在本项目中，Workflow 被定义为多个 Agent 围绕同一任务形成的内核级协作单元。其核心标识包括：`workflow_id`，用于标识工作流身份；`workflow_leader_pid` 与 `workflow_leader_generation`，用于标识领导者及其身份代次；`agent_parent_pid` 与 `agent_parent_generation`，用于表示父子协作链路；`agent_global.workflows[]` 则统一维护全局工作流状态。系统通过这些字段将协作关系显式写入 PCB 和全局结构中，使工作流首次成为内核能够识别、调度和治理的对象。

围绕这一结构，系统实现了完整的工作流生命周期管理链路。

`agent_workflow_create_locked()` 用于创建工作流条目，`agent_workflow_join_locked()` 负责新成员加入，`agent_workflow_leave()` 处理成员退出，而 `agent_cascade_terminate()` 则用于执行级联终止。子 Agent 创建时会自动继承并归属到同一工作流中，内核统一记录 Leader、成员关系和运行状态，避免用户态自行维护复杂的协作绑定关系。

该机制最具创新性的部分在于级联终止治理。传统 xv6 中，父进程退出后仅执行 `reparent()`，并不理解这些子进程可能属于同一任务工作流，因此很容易遗留正在 `agent_wait()` 中睡眠的孤儿 Worker。AgentOS 则在主控进程正常退出、异常崩溃或主动下发终止指令时，由内核遍历同一 Workflow 的全部成员，停止其心跳、清空事件监听、注入 `PARENT_GONE` 事件，并主动唤醒阻塞中的 Agent，使其统一进入终止收敛路径。这样，系统可以彻底避免“主控已退出但 Worker 永久休眠”的失控问题。

同时，系统通过 `identity_generation` 与 `workflow_leader_generation` 两级代次信息，解决了 PID 复用下成员误认与领导者身份混淆的问题。也就是说，系统在判断“谁属于这个 Workflow”“谁是它的 Leader”时，不再仅依赖 PID，而是依赖“PID + 代次”的复合身份，从而显著增强了工作流治理的可靠性。

在此基础上，AgentOS 进一步实现了 Workflow 级指标汇总能力。系统不再只以单个进程为观测单位，而是首次能够以内核原生工作流为单位，聚合统

计工具调用次数、文件查询次数、消息交互次数、等待行为等运行指标。这意味着内核可以直接观察“一个任务”的整体执行负载，而不是要求用户态再把多个 Agent 的日志手工汇总。由此，Workflow 不仅成为可治理的协作边界，也成为可观测的任务统计边界。

因此，内核原生 Workflow 治理的创新意义在于：它将多 Agent 协作关系从临时消息绑定提升为系统级组织对象，并通过代次身份、级联终止和任务级指标汇总，使内核第一次能够以“任务”而非“单进程”为单位对 Agent 工作负载进行统一管控和观测。这是 AgentOS 区别于传统通用进程模型的重要标志之一。

2.4.5 创新五：多 Agent 安全管控体系

现有面向 AI Agent 的安全方案多部署于应用层，内核无法拦截越权操作，权限、数据、并发修改存在多重安全盲区。AgentOS 在内核构建一套分层协同安全机制，覆盖身份、权限、上下文、文件协作、进程协作全场景，是区别通用 xv6 内核的关键创新。

采用角色 + Capability 位图双层权限模型，Planner、Patch 等角色预设专属能力模板，运行时仅支持降权、禁止进程自提升权限，所有工具调用统一校验能力位，越权操作留存审计日志。

对于内存文件实现多重数据防护，Context 通过 Guard 内存保护页、奇偶快照代数、链式摘要三层机制，拦截越界读写、半写脏读与事后篡改；文件修改依托 inode 排他租约，防止并发冲突与非法覆盖。

同时保证进程安全。依靠 identity_generation 身份代次消除 PID 复用漏洞，租约、工作流、消息归属校验同时匹配 PID 与代次，避免进程复用带来权限误绑定。配套 Workflow 域隔离，动态工具、查询缓存仅同工作组可共享，跨组服务需标记公开；fork 设计保守继承策略，子进程仅继承 Agent 身份，重置角色与权限，阻断权限传递泄露。整套机制无需重型加密，轻量化嵌入内核执行链路，所有高危操作可拦截、可审计、可追溯，在不牺牲多 Agent 协作灵活性的前提下，构建完整内核安全隔离底座。

3 详细设计与实现

本节介绍项目的详细设计与主要实现细节。

3.1 Agent 进程结构与生命周期设计

3.1.1 关键结构

为了让进程能够承载多轮工具调用、上下文保留、事件等待、多 Agent 协作和权限治理，系统在不推翻 xv6 基础框架的前提下，将普通进程扩展为 Agent 进程。因此，当前 Agent-OS 通过补充 Agent 字段，使进程获得了新的语义能力。Agent 不是一个抽象概念，而是内核中可被识别、可被调度、可被约束的具体进程类型。

普通进程与 Agent 进程的主要差异如下：

比较项	普通进程	Agent 进程
身份标记	agent_type=AGENT_TYPE_NORMAL	PRIMARY 或 WORKER
角色	无	Planner / Retriever / Patch / Test / Reviewer / Tool Service
Context 区	无	有专用用户态 Context 区
心跳与事件	无	可配置心跳、监听消息/文件修改等事件
权限模型	仅依赖传统系统调用边界	角色 + capability 位图
消息机制	无语义邮箱	有界 FIFO mailbox
工作流	无	workflow id / leader / member / cascade kill
文件协作	普通文件操作	附加 lease、query_file、文件事件通知

3.1.2 关键函数

- **fork 继承**

如果在 fork 后完全保留父进程的 Agent 运行状态，会出现几个问题：

- 子进程会错误继承父进程历史 Context 轨迹

- 子进程会共享旧 heartbeat 与等待状态
- 子进程会复用旧 identity，导致 PID/身份混淆
- workflow 成员统计可能失真

因此，当前 fork 实现选择继承 agent 的身份基础，但重置运行态数据。通过该设计，在进 Agent 派生时，子进程仍是 Agent，但需要作为新的执行实体重新进入运行，而不会发生冲突。

● exit 清理路径

Exit 单个进程时，其其主要负责包括：

- 将进程从心跳时间轮移除
- 清理动态工具注册信息
- 清理动态请求槽
- 回收该 PID 持有的 lease
- 退出 workflow 成员关系

它是 Agent 生命周期结束时的资源回收器。

但是，在多个 Agent 进程合作时，仅仅终止当下进程，有可能导致僵尸进程。因此需要扩展 exit 逻辑。agent_cascade_terminate()函数用于工作流级联终止，其处理思路是：

1. 验证 leader 身份
2. 读取 workflow_id 和 leader_generation
3. 将 workflow 状态切换为 terminating
4. 找到同一 workflow 的 Agent
5. 将其 loop_state 置为 DONE
6. 注入 PARENT_GONE 或终止语义

这保证了多 Agent 协作任务不会因为 leader 退出而遗留失控 worker。

3.1.3 workflow 与多 agent 协作

多 Agent 协作任务不应仅靠消息松散连接。内核需要知道：哪些 Agent 属于同一个任务流；谁是 leader；当前 workflow 是否还处于 active 状态；leader 退出时是否需要级联结束。

因此系统引入了 workflow 结构。若创建的是 PRIMARY Agent，则默认成为 workflow leader。若是 worker，如果已有 workflow 信息，则尝试加；如果 workflow 信息，则也可单独创建自己的 workflow。

通过 workflow，项目对主控 Agent 和工作 Agent 两种角色进行区分。

当 Agent 退出或注销时，该函数会找到所属 workflow，将成员计数减一，若成员数归零，则将 workflow 标记为 dead 并清空。
workflow 生命周期与成员存活状态进行持续绑定。

3.2 Context 上下文系统

3.2.1 关键结构

```

01 // 因 Context 容量不足而被 FIFO 淘汰的节点总数
02 uint64 context_dropped_nodes;
03
04 // 每个 Context 节点在 Path 区中的起始偏移
05 uint16 context_offsets[AGENT_CONTEXT_MAX_NODES];
06
07 // 每个 Context 节点在 Path 区中的记录长度
08 uint16 context_lengths[AGENT_CONTEXT_MAX_NODES];
09
10 // Context 快照代数，用于区分“写入中”和“稳定快照”
11 uint64 context_generation;
12
13 // 当前 Context 窗口中最早一条节点的 sequence
14 uint64 context_first_sequence;
15
16 // 下一条待写入节点的 sequence
17 uint64 context_next_sequence;
18
19 // 内核侧维护的 Context 可信摘要环
20 struct agent_context_digest context_digests[AGENT_CONTEXT_DIGEST_MAX];
21
22 // 摘要环当前写指针
23 int context_digest_head;
24
25 // 摘要环中当前有效摘要条数
26 int context_digest_count;
27
28 // 最近一次 Context 查询/复用的追踪标记
29 uint64 last_context_query;
30
31 // 是否发生过 Context 复用命中
32 int context_reuse_hit;

```

在 Context 结构设计中，Context 不只是用户态的一块内存，内核还维护了一套并行元数据索引。整合节点淘汰统计、路径位置偏移、分段长度、全局序列号、快照版本代数、内核可信摘要环形缓冲区、上下文复用观测标记等全套管控字段。支撑路径节点快速定位、有序写入、容量 FIFO 淘汰逻辑，保障海量路径节点高效存取。依靠快照代数区分不同写入阶段的上下文状态，通过循

环摘要环实现内核层数据完整性可信校验，并持续统计上下文复用命中情况、记录最新查询标记。

`agent_context_node` 是一次上下文记录的逻辑输入单元，用于记录一次工具调用或一次交互事件的完整上下文语义。其中最关键的不是 `request/result` 本身，而是这几个关系字段：

- `sequence` 表示上下文时间序。
- `request_id` 表示一次调用实例。
- `span_id` 用于跨调用链追踪。
- `cause_sequence` 表示本节点可能由哪条历史上下文触发。

`agent_context_node` 在记录交互结果的同时，也一次 `Agent` 行为抽象为带时序和关联关系的事件节点并保存。通过上述关系字段，`Context` 不仅能保存执行结果，还能表达调用链与因果链，从而具备轻量级行为追踪能力。

用户态 `context` 结构主要由三部分构成：

- `Head`
- `Path`
- `Guard page`

分配用户态 `Context` 内存区域时，会在整块上下文存储空间的末尾额外分配一页专用 `Guard` 保护页，并通过内核内存权限接口清除该页面的用户读写访问权限。该保护页不会承载任何上下文文本、元数据或摘要信息，仅作为内存访问边界隔离屏障。当 `Agent` 程序对 `Context` 区进行超范围读写时，访问操作会直接命中无权限 `Guard` 页，触发内核内存异常中断，及时捕获非法越界行为，避免程序静默篡改上下文 `Header`、`Path` 日志与内核元数据，提前阻断因越界覆盖引发的一致性故障，提供基础安全隔离防护。

3.2.2 Path 结构

当前 `Context` 存储并未采用独立结构体数组存储单条执行节点，而是采用固定 `Header` 元数据区 + 连续变长文本日志区的混合存储架构。系统写入每条 `context_node` 时，会将时序、调用链路、请求响应等全部结构化字段序列化统一格式的单行文本记录：

```
{ts=...,seq=...,req_id=...,span=...,cause=...,status=...,req=...,res=...}
```

每条记录以换行结束，这是一种文本日志 + 内核索引的混合设计。

使得用户态可直接读取整块 `Path` 文本区，无需解析复杂二进制结构体，通过字符串检索就能快速定位指定时序、请求 `ID` 的工具调用记录，日志可读性强。同时使上下文回滚逻辑轻量化，执行 `context_rollback` 仅需根据索引截断

文本尾部，不用对复杂对象做序列化 / 反序列化销毁，大幅简化内核裁剪逻辑。由于上层 Agent 推理天然适配文本输入，LLM 策略层可直接读取整块文本作为对话与执行历史，无需内核额外转换数据格式，省去二进制与文本互相转译的开销。

3.2.3 核心函数

● agent_context_push_node

这是 Context 设计中最核心的写入函数，负责把逻辑节点写成真实上下文。

在 agent_context_push_node 的实现中，一次上下文节点写入并不会只更新单一存储区域，而是同时写入两类语义不同的数据副本，进行“双写”。“双写”并非指数据被复制两次，而是指同一条上下文事件会以两种不同形式分别进入用户态可见的上下文文本区和内核态维护的可信摘要环。

第一类写入发生在用户态可见的 Context Path 区。函数首先将 agent_context_node 中的 timestamp、sequence、request_id、status、request 和 result 等信息序列化为一条文本记录，写入 Path 区尾部。向用户态 Agent 提供可直接读取的上下文内容，使其能够在后续推理、回滚、摘要生成或协作过程中复用已有历史。

第二类写入发生在内核维护的 context_digests[] 摘要环中。完成文本记录写入后，系统会调用 agent_context_digest_push()，对当前节点的 request、result、status、sequence 和 request_id 等关键信息计算摘要，并与前一条摘要的 current_hash 链接，形成链式可信记录。该摘要环并不保存完整文本，而是保存压缩后的结构化校验信息，其主要职责不是供用户态阅读，而是用于后续一致性验证、篡改检测和上下文历史链完整性检查。

这两类写入虽然来源于同一条 agent_context_node，但面向的目标完全不同。前者构成 Agent 可见的 Context 历史，后者构成内核可信的 Context 校验链，二者共同保证了 Context 子系统既可使用，又可验证。

● agent_context_query

agent_context_query 的行为相对简单，从 Path 区拷贝最多匹配的内容到用户缓冲区，并在查询成功时设置 context_reuse_hit = 1。

Context 查询不仅是读操作，还将 Context 查询视为一次上下文被重新利用的行为标记。只要 Agent 主动查询自身已有 Context，就意味着历史上下文已经参与到后续执行过程之中，因此这次读取在语义上可以被视为一次 Context 复用命中。

系统通过 `context_reuse_hit` 这一标记，可以在后续统计、调度优化或上下文管理策略中识别出哪些 Agent 确实在利用历史上下文。若某一类 Agent 长期频繁触发 Context 查询，则说明该 Agent 对上下文依赖较强，未来可以考虑为其分配更大的 Context 配额，或在摘要压缩、淘汰策略和缓存策略上给予更高优先级。反之，若某些 Agent 很少查询历史 Context，则说明其行为模式更接近一次性执行体，对长期上下文窗口的需求较低。用命中”观测语义，用于后续统计或优化。

- **agent_sync_header / agent_context_write_begin**

这两个函数体现了 Context 快照的一致性策略：

Agent 上下文更新为多步串行非原子操作，直接读取易出现头部元数据与路径数据不匹配的半写脏数据，因此系统摒弃重型锁机制，通过 `context_generation` 奇偶值区分快照状态。

`agent_context_write_begin` 启动上下文写入时，将该世代值置为奇数，标记系统处于写入中间态，此时数据未同步完成，不可信读取。待所有数据写入完毕后，`agent_sync_header` 将世代值更新为偶数并固化头部信息，标识快照稳定完整。用户态读取时需校验世代值，奇数需延迟重试，偶数即可安全读取。该机制兼顾数据一致性与性能开销，高效保障了 Agent 上下文数据的完整可靠。

3.2.4 淘汰算法-FIFO

系统为每个 Agent 的 Context 设置双重容量上限，节点总数上限与总文本上限，触发任一阈值便持续执行淘汰逻辑。

淘汰函数执行 FIFO 完整移位逻辑。读取最早第一条文本节点的字节长度，将整条 Path 区内后续全部日志内容向前平移覆盖旧记录，同步修正内核索引数组中所有节点偏移量，统一左移对应长度。全局节点计数 `context_node_count` 自减，淘汰统计 `context_dropped_nodes` 加一，同时更新窗口起始序列号。

3.2.5 摘要校验与一致性设计

- **digest push 机制与 digest verify 机制**

每条上下文节点写入完成后，内核调用 `agent_context_digest_push()` 生成校验指纹。函数不对完整请求、响应文本进行存储，仅分别计算二者哈希摘要以缩减存储开销。同时读取上一节点指纹作为当前节点前驱哈希，将序列号、请求标识、内容哈希、前驱指纹与运行状态聚合，生成本条节点最终指纹。

各节点指纹依次依托前驱哈希形成强依赖链式结构，若对任意中间记录执行删除、插入、替换操作，整条后续哈希链路将同步失效，从底层阻断上下文非法篡改、拼接行为。

系统通过摘要推送与摘要校验机制，构建 Agent 上下文的链式可信校验能力。其中，`agent_context_digest_push()` 使每条记录均关联上一节点哈希，形成环环相扣的摘要链，避免数据被随意篡改或非法拼接。对应的 `agent_context_digest_verify()` 负责上下文完整性校验，依次校验摘要总数与链表头合法性、前后哈希匹配关系、序列号递增规则及单条哈希一致性，任意校验项异常即判定上下文损坏或被篡改。系统构建了双层一致性保障体系

摘要链机制负责全局历史数据完整性校验，抵御事后篡改、删除与链路断裂风险。

3.3 Tool Call 与进程互动

该模块主要负责 Agent-OS 中工具调用框架、内建工具分发、动态工具注册、消息通信与 Agent 发现协作机制。当前实现中的 Tool 并不是用户态脚本层概念，而是内核显式调度、校验权限、记录上下文并支持服务化扩展的一类标准能力接口。

工具由两大部分构成：内建工具、可动态注册的工具。工具调用子系统主要解决以下问题：

- 为 Agent 提供统一接口调用。
- 将文件、上下文、消息、查询、租约等能力抽象成标准工具。
- 在调用前执行 `capability` 权限校验。
- 支持除了内建工具外的动态工具注册与服务化调用。
- 支持 Agent 之间的消息互动与发现协作。
- 将工具调用轨迹自动写入 Context，形成可追踪执行历史。

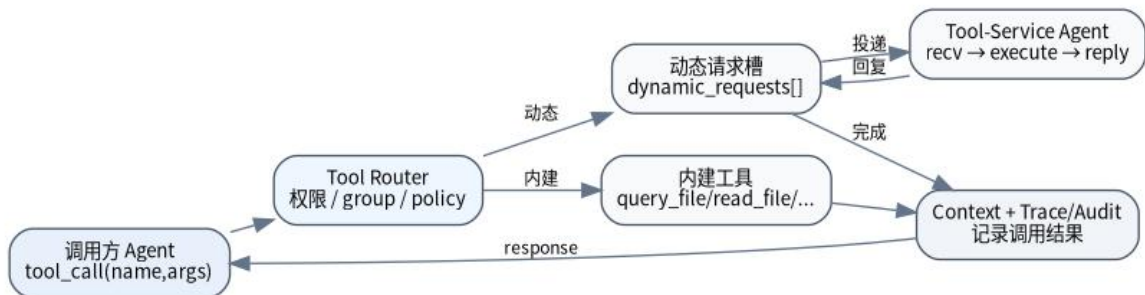


图 3.3 工具调用流程图

3.3.1 内建工具

本项目没有使用直接暴露内核函数的松散调用模式，而是构建权限约束、统一调度、自动留痕的标准化工具执行框架，依托工具策略表与统一调用入口实现全链路管控。

- 内建工具策略表 `builtin_policies[]`

该数组并非简单工具名录，而是内核能力治理元数据表，为每款内置工具绑定多维管控属性：工具名、调用必需能力位图 `required_cap`、协作分组约束 `group_policy`、状态修改标识 `mutates_state`、审计标记 `audit_required`。系统可在调用前置校验权限、判定状态变更风险、确认审计需求、限定访问域，将零散内核函数升级为带完整治理规则的标准化能力目录，实现工具行为可预管控。

- 统一调用入口 `agent_tool_call()`

作为所有工具的唯一执行入口，该函数融合鉴权、状态机切换、日志沉淀全流程逻辑。首先校验调用者 Agent 身份，基于策略表统一校验 Capability 权限；切换 Agent 运行状态为 RUNNING；路由分发执行对应工具逻辑；调用结束自动构造上下文节点并写入 Context 持久化轨迹；最终将运行状态切回 READY。单次工具调用天然绑定权限拦截、生命周期流转、执行记录留存三类系统行为，将分散治理逻辑收敛至同一执行链路。

● 批量调用接口 agent_tool_call_batch()

该接口为单条调用的顺序封装，仅聚合多份请求参数，不提供原子事务保障。批量内每条子请求独立完成权限校验、独立生成上下文记录；若中途调用失败，已执行操作不会自动回滚，仅简化上层批量发起流程，无事务一致性语义。

部分内建函数功能如下：

分类	工具函数名	功能说明
	tool_get_system_status()	返回进程数、Agent 数、ticks、心跳统计等全局系统状态
查询类	tool_query_process()	按条件列出进程或 Agent
	tool_query_agent()	按 role/capability/group 查询 Agent 信息
	tool_read_context()	读取当前进程最近的上下文文本
	tool_read_file()	读取文件内容
	tool_patch_file()	进行替换式补丁写入
文件相关	tool_diff_file()	输出简化差异信息
	agentfs_tool_set/get/del/query_file()	通过 AgentFS 进行文件属性设置、获取、删除与检索操作
协作类	tool_send_message_tool()	内部调用 agent_send_message(), 向目标 Agent 发送消息

	<code>tool_query_agent()</code>	执行 Agent 发现, 查找定位目标 Agent
	<code>tool_lease_begin/commit/abort()</code>	进行文件协作租约控制, 支持开始、提交、中止三种操作
测试辅助	<code>tool_run_rule_test()</code>	运行规则式检查, 用于测试验证

3.3.2 动态工具注册与调用

● 注册

为突破内建工具集合固定、功能扩展能力有限的问题, 系统引入了动态工具注册机制。该机制的核心数据结构包括服务注册表 `dynamic_tools[]` 与 调用转发队列 `dynamic_requests[]` 两部分。

其中, `dynamic_tools[]` 用于保存当前系统中已经注册的动态工具项。它并不是简单的名称列表, 而是一个同时承载所有权、作用域、可见性、权限需求的运行时服务目录。

每个表项本质上对应一个由 Agent 进程主动对外发布的服务端点, 其核心字段包括:

- `used` 用于标识该表项当前是否有效;
- `name` 用于唯一标识工具名称;
- `owner_pid` 表示该动态工具由哪个 Agent 进程拥有和提供;
- `owner_group` 用于限定该工具的协作作用域;
- `flags` 用于描述工具是否允许跨组公开访问等可见性属性;
- `required_cap` 则用于表达调用该工具所需具备的能力约束。

在此基础上, `agent_tool_register()` 负责完成动态工具的注册。该函数并不允许任意进程随意扩展工具空间, 而是设置了权限、命名、可见范围等若干明确约束。

动态工具并不是内核直接实现的新函数, 而是由 Agent 进程在运行期发布出来、并由内核登记和调度的服务端点。也就是说, 动态工具本质上实现了一种进程服务化思想。Agent 不仅可以消费工具, 还可以成为工具提供者。

● 调用

动态工具调用并非采用直接函数跳转或同步进入目标进程上下文的方式完成, 而是通过 `dynamic_requests[]` 请求槽数组进行中转。系统先将请求写入一

个内核维护的请求槽，再由目标 Agent 主动取走并处理。使得动态工具调用实现了了消息转发与服务解耦。

每个请求槽本质上描述一条尚未完成的动态工具调用，它记录了调用者 PID、服务者 PID、调用组、请求参数、结果内容、回复状态、是否已投递以及是否已回复等信息。

调用方只负责把请求投递到队列中，服务方则从队列中取走请求并写回结果。这种机制避免了调用方与服务方在执行时直接耦合，也使内核能够在两者之间插入权限控制、状态管理和异常清理逻辑。

这相当于在内核里实现了一层轻量 RPC。动态工具的实际调用流程由 `agent_dynamic_tool_call()` 主导，在本地内核中模拟了“请求发起方 - 服务提供方 - 中转调度层”这一经典结构。

首先，调用方提交工具名和参数后，内核会在 `dynamic_tools[]` 中查找是否存在同名动态工具，以确定本次请求的目标服务者。找到目标后，系统在权限校验通过之后，内核会尝试为当前请求分配一个空的请求槽。若没有空槽，则说明动态请求队列已满，系统会直接返回忙状态而不是继续阻塞扩容。分配到空槽后，内核会将工具名、参数、调用者 PID、服务者 PID 等信息写入该槽，并调用 `wakeup()` 唤醒正在等待动态请求的服务 Agent。至此，请求已经完成投递。

接下来，调用方并不会继续向前执行，而是进入睡眠状态，等待该请求槽被服务方处理并回填结果。服务方随后通过 `agent_tool_recv()` 从请求槽数组中取出属于自己的待处理请求；在用户态或服务逻辑中完成处理后，再通过 `agent_tool_reply()` 将结果字符串与状态码写回原请求槽，并标记为已回复。此时内核会再次唤醒等待中的调用方。调用方被唤醒后，会读取结果、构造响应并清理该请求槽，至此一次动态工具调用才真正完成。

● 注销

为了保证动态工具系统不会因服务进程退出而留下无效注册项或悬空请求，系统在 `agent_proc_exit()` 中实现了完整的清理逻辑。动态工具严格依附于其拥有者进程。一旦服务进程退出，内核就必须同步回收与其相关的所有运行时状态。

首先，删除该进程在 `dynamic_tools[]` 中注册的所有动态工具项。防止后续调用者继续查询到一个实际上已经不存在的服务端点。而后，清理所有“以该进程为调用者”的请求槽，这些请求已经失去原始发起者，继续保留没有意义。对于以该进程为服务者但尚未回复的请求，系统不会无声丢弃，而是将其状态显式标记为 `SERVICE_GONE`，并写入 `tool service exited` 的错误结果，从

而让等待中的调用方能够得知失败原因。最后，内核会唤醒所有相关等待方，使其不至于因为服务方已经退出而永久阻塞。

这一清理机制防止了注册悬空与请求悬空，即调用方仍然能看到一个实际上已经不存在的工具、调用方永远等待一个再也不会回复的服务进程。确保了动态工具与进程生命周期之间的一致绑定关系。

3.4 AgentFS 文件查询与协作

AgentFS 并不是重写 xv6 文件系统，而是在原有 inode/目录体系上增加属性、摘要、索引、缓存与协作控制，从而让文件系统具备可查询、可协作、可感知的 Agent 语义，主要为 Agent-OS 提供以下几个功能：

- 让 Agent 能按属性和摘要查询文件，而不是只能按路径访问。
- 给文件附加结构化属性，便于角色化检索。
- 为查询结果构建索引和共享缓存，降低重复扫描成本。
- 为并发修改提供租约保护机制，降低写冲突。
- 在文件变化时主动通知相关 Agent。

3.4.1 AgentFS 结构扩展

在 inode 扩展设计中，添加的最关键的元素如下：

- `attrs[]` 用于为文件附加结构化键值属性，使文件能够支持基于语义标签的条件检索；
- `attr_count` 用于描述当前有效属性数量，保证属性集合的边界清晰可管理；
- `summary` 则保存文件内容的摘要或前缀文本，用于支持 keyword 级别的轻量文本匹配。

扩展 inode 后，文件不仅具有路径语义，还具备可检索的结构化语义与文本语义。

同时，在 `agent_file_meta` 缓存设计中添加了类似的扩展，把分散在 inode 中的文件语义统一提取出来，形成便于索引、过滤与缓存复用的元数据基础：

- `dev`、`inum` 共同标识文件的内核身份，
- `path` 用于向 Agent 返回可直接操作的文件路径，
- `attrs[]`、`summary` 则将文件的属性信息与文本特征整理为内存中的查询镜像。

3.4.2 索引设计

该索引采用三层数据结构组织文件与属性之间的映射关系：

- `file_meta[]`：文件元数据总表，每个数组元素对应一个文件的元数据镜像，存储文件的完整属性信息。

- `file_postings[]`: 属性到文件的连接表，记录特定属性条件所关联的文件条目，构成倒排索引的 `posting` 链。
- `file_index[]`: `posting` 链的桶入口数组，相当于哈希目录，通过哈希函数将属性键值映射到对应桶，实现快速定位。

`agent_file_posting` 作为 `posting` 链的基本节点，包含以下关键字段：

- `key`: 属性名称，例如 `type`;
- `value`: 属性值，例如 `code`;
- `entry_idx`: 指向 `file_meta[]` 中对应文件条目的索引;
- `next`: 指向同一哈希桶中下一个 `posting` 节点的指针。

该结构本质上表达了“属性条件 `key=value` 对应哪些文件”的映射关系。若多个文件均携带 `type=code` 属性，则索引中存在多条 `posting` 节点，分别指向不同的 `entry_idx`。实现了轻量级倒排索引，其设计思想与搜索引擎中关键词到文档集合的映射机制一致，仅将映射对象由文档替换为文件。

3.4.3 文件查询机制

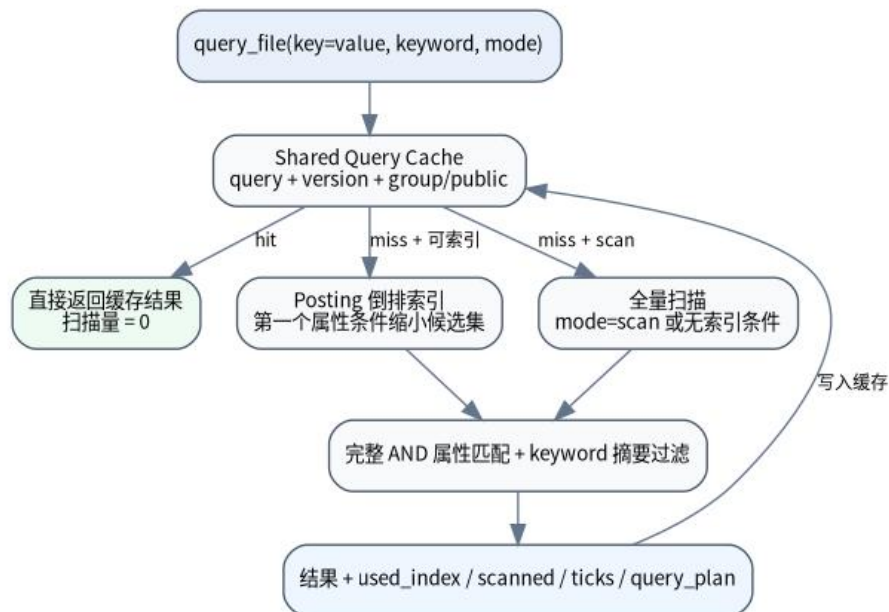


图 3.4 文件查询流程图

`query_file` 支持的是属性检索 + 摘要关键字过滤的组合模式。当前实现通过 `key=value` 的属性条件与 `summary` 关键字过滤相结合，使文件能够按照“类型、归属、标签、状态、内容特征”等语义信息被检索。也就是说，系统不再仅仅能够按路径查询，也能够处理哪些文件满足某类任务属性，并且具有某种内容特征。这种查询方式更符合 Agent 在协作执行中的实际需求，因为 Agent

在多数场景下关注的并不是静态路径，而是文件所承载的功能角色与任务语义。

查询执行采用先缩小候选集、再精确过滤的两阶段策略。当查询包含至少一个属性条件且未显式指定 `mode=scan` 时，系统选取第一个属性条件作为哈希入口，在 `file_index[]` 对应桶中定位 `posting` 链，仅遍历该链所关联的候选文件子集，随后调用 `file_meta_matches()` 对剩余属性条件及关键词进行完整校验。

该策略的核心在于，哈希桶结果仅作为候选集入口，而非最终查询结果。第一阶段通过索引粗筛大幅缩减待检查文件数量，第二阶段对候选集执行逐条精确匹配，确保查询结果的准确性。这种粗筛与精筛相结合的模式，在实现复杂度与查询性能之间取得了较好的平衡。

3.4.4 共享缓存机制

共享缓存的生成具有被动触发特性，仅在首个 Agent 发起全新文件查询后产生。当 Agent 调用 `query_file` 接口且未命中已有缓存时，系统将完整执行查询流程，包括查询参数解析、索引检索或目录扫描、查询结果封装等操作。查询完成后，系统通过 `shared_query_cache_store()` 函数将本次查询的完整结果持久化存入 `shared_query_cache[]` 缓存数组，同时记录关键溯源与校验信息，包含唯一查询字符串、结构化查询结果、创建进程 PID、所属协作组以及当前文件系统版本。该设计保证仅已执行的有效查询才会生成可共享缓存，从源头保障缓存数据的有效性。

后续其他 Agent 发起相同文件查询请求时，系统不会直接执行检索逻辑，而是优先通过 `shared_query_cache_lookup()` 函数执行缓存命中校验。复用生效需同时满足三项条件：查询字符串完全一致，保证查询需求完全相同；缓存绑定的文件系统版本与当前系统版本匹配，确保文件状态未发生变更、结果未失效；当前访问 Agent 具备对应缓存的复用权限。全部条件校验通过后，Agent 可直接复用已有查询结果，无需重复遍历文件系统与执行索引检索，大幅简化查询流程、降低系统开销。

3.4.5 权限管控

系统通过 `cache_access_allowed()` 函数实现精细化的缓存共享权限管控，杜绝无权限 Agent 随意复用查询结果，构建分层可控的共享体系。权限规则分为两类：

公开共享，当查询参数配置 `public=true` 或查询归属为系统级任务 `owner=system` 时，缓存结果对所有合规 Agent 开放。

其二为组内私有共享，默认情况下，缓存结果仅允许同一协作组内的 Agent 复用，跨组 Agent 无法获取缓存数据。

该权限机制精准适配多 Agent 分组协作的运行模式，兼顾缓存复用效率与数据访问安全性。

3.4.6 文件协作与租约机制

● 机制设计目标

在多 Agent 协同开发场景中，多个 Agent 可能并发对同一文件执行修改操作，极易出现内容覆盖、提交冲突与数据错乱等问题。因此系统引入轻量级文件租约机制。

Agent 执行文件修改前需主动申请修改租约，获取临时独占操作权限；修改完成后主动释放租约。针对异常未释放的租约，系统通过超时策略自动回收，从运行秩序层面规范多 Agent 的文件修改行为，保障并发修改的安全性。

`agent_proc_lease_begin()` 用于 Agent 发起文件修改租约申请，获取文件的临时独占修改资格。首先校验调用者权限，仅具备 `AGENT_CAP_LEASE_ACQUIRE` 或补丁修改能力的 Agent 可发起申请；而后通过路径解析获取文件 `inode` 与唯一设备标识；随后周期性清理系统内已过期的无效租约，避免资源占用；再校验目标文件是否存在未过期租约，若已被占用则直接返回繁忙状态，拒绝本次申请；无冲突时分配空闲租约槽位，写入文件标识、持有者信息、版本信息与过期时间，最终返回租约编号与文件基础版本。系统采用排他式租约设计，同一时刻仅允许单个 Agent 持有同一文件的修改租约，从根源杜绝并发修改冲突。

● 租约释放机制 (`lease_commit` / `lease_abort`)

系统提供两种语义不同的租约释放方式，适配不同执行场景。其中 `agent_proc_lease_commit()` 为正常结束释放，适用于 Agent 完成全部文件修改流程的场景，函数会校验租约编号、持有者身份、代次信息与租约有效性，校验通过后清空租约记录。当前版本的版本校验逻辑为轻量化预留设计，暂未实现严格的乐观并发校验，核心作用为合法释放文件独占修改权限。

`agent_proc_lease_abort()` 为主动放弃释放，适用于 Agent 获取租约后终止修改任务的场景，可主动归还独占权限，避免无效占用文件资源，保障其他 Agent 可正常发起租约申请。

为解决 Agent 异常退出、进程卡死、未主动释放租约导致的文件假占用问题，系统设计自动回收策略。

定时过期回收，`agent_lease_reap_expired()` 依托系统时钟周期执行，自动清理所有超时未释放的租约，避免资源永久占用；

进程生命周期联动回收，当 Agent 进程退出时，`agent_lease_reap_pid()` 会主动清空该进程持有的全部租约。

3.4.7 文件变更事件通知机制

租约机制解决了多 Agent 文件并发修改的权限冲突问题，规范了文件修改的前置秩序。为解决文件修改后的跨 Agent 协同联动问题，系统配套设计文件变更通知机制，实现文件变更事件的精准、高效传播，保障协作 Agent 及时感知文件状态变化。

系统区分粗粒度广播通知与细粒度精准通知，适配不同 Agent 的监听需求。第一层为全局广播通知，由 `agent_signal_filemod()` 实现，面向所有订阅 FILEMOD 事件的 Agent 推送变更通知，适用于仅需感知全局文件系统变化、无需关注具体文件的协作场景。第二层为精准定向通知，由 `agent_notify_file_modified(dev, inum)` 实现，仅向监听指定 inode 文件的 Agent 推送变更消息，精准匹配目标监听对象，减少无效事件推送，提升系统协作效率。

Agent 可通过 `agent_proc_watch_file()` 注册指定文件监听任务。该接口通过路径解析获取目标文件 inode，记录文件设备标识、inode 编号与文件路径，并开启文件变更监听权限。当文件发生变更时，系统通过匹配 dev 与 inum 标识，精准定位对应监听 Agent，置位文件变更事件标记、更新唤醒时间戳，将处于睡眠状态的 Agent 切换为可运行状态，实现文件变更与 Agent 调度的联动响应。

系统的文件变更通知不仅覆盖文件内容修改，同时兼容文件元数据与属性变更。`patch_file`、`set_file_attr`、`del_file_attr`、`agentfs_content_changed()` 等操作均会触发变更通知，可全面感知文件内容、类型、归属、状态等核心信息的变化，全方位支撑多 Agent 基于文件状态的协同工作。

3.5 Agent Loop、心跳与事件感知调度

这一模块的核心目标是让 Agent 不再只是被动占用 CPU 的普通进程，而是成为一个能够事件驱动、周期唤醒、带预算运行、可优先级调度的协作执行体。

状态机

loop_state 用于描述 Agent 当前所处阶段。结合代码可以看到主要状态包括：

- AGENT_LOOP_IDLE 尚未进入有效 Agent 循环
- AGENT_LOOP_READY 已就绪等待运行
- AGENT_LOOP_RUNNING 正在执行工具或逻辑
- AGENT_LOOP_WAITING 正在等待消息/文件/心跳/超时
- AGENT_LOOP_ROLLED_BACK 执行了上下文回滚
- AGENT_LOOP_DONE 已结束循环

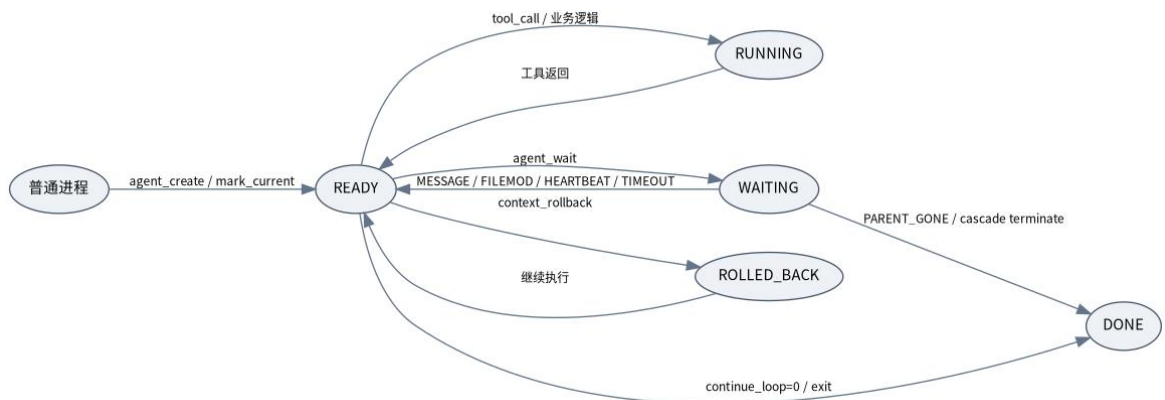


图 3.5 Loop 状态机

3.5.1 心跳机制

为突破纯事件驱动的限制性，系统引入心跳机制，赋予 Agent 无外部事件亦可自主周期唤醒的能力，解决被动监听模式下无法主动轮询、周期性自检、超时巡检的问题。同时该机制配合调度优化策略，避免系统每一时钟周期全量遍历所有 Agent，大幅降低内核调度开销。

心跳机制依托进程级与全局两级字段实现精细化管控。进程级字段包含心跳周期、下次唤醒截止时间、时间轮挂载位置与活跃状态，用于定义单 Agent 的唤醒规则与挂载状态；全局维护心跳链表、统计计数、最小截止时间与扫描统计变量，统筹管理全系统心跳任务。

系统提供标准化接口实现心跳启停管控。`agent_proc_heartbeat_set()` 用于启用周期心跳，配置唤醒周期、计算截止时间、清除历史无效状态，将 Agent 重置为就绪态并重新挂载至时间轮；`agent_proc_heartbeat_stop()` 用于终止心跳任务，将 Agent 从时间轮摘除、清空周期配置与状态标记，彻底终止周期性唤醒能力。

3.5.2 时间轮设计

常规心跳机制实现方式为每次时钟中断遍历全部 Agent，逐一比对心跳截止时间。该方案实现简单，但存在显著性能缺陷。绝大多数 Agent 在单个时钟周期内不会触发心跳，持续产生无效判断。随着 Agent 规模增长，全量扫描的时间开销持续上升。

时间轮针对该问题优化，核心思路为：

- 将 Agent 唤醒截止时间离散映射至指定桶；
- 每个时钟周期仅处理当前时刻匹配的桶；
- 周期任务触发完成后，重新规划并插入未来对应桶。

系统不再轮询全部 Agent 检索到期对象，而是预先登记任务预期触发时刻，到达时间点仅处理候选集合，收缩检查范围，有效削减周期性心跳调度成本。

时间轮状态分为 Agent 私有状态与全局管理状态两层：Agent 本地状态、全局 `agent_global` 管理状态。

整体结构可概括为全局桶数组，每个桶维护一条 Agent 链表，各 Agent 记录自身所属桶信息。

● 任务插入机制

配置 Agent 心跳周期或重新规划唤醒时机时，调用 `agent_heartbeat_wheel_insert_locked()` 完成注册，通过 `heartbeat_deadline` 与掩码运算计算目标桶号，利用低位映射实现时间向循环桶数组的映射；

将 Agent 挂载至对应桶链表头部，依靠链表字段串联桶内多个 Agent；

更新 Agent 激活标识、所属桶编号，同步修改全局任务计数；若当前任务截止时间早于全局最小截止时间，则更新全局阈值。

插入操作仅完成任务预登记，代表 Agent 心跳将在未来对应时刻参与调度，不会立即触发唤醒。

Agent 停用心跳、退出执行或调整调度策略时，调用移除接口将节点从时间轮摘除。执行逻辑为定位 Agent 所在桶，遍历链表找到目标节点并脱离链表；清空 Agent 本地时间轮相关标识，同步减少全局活跃任务计数。

移除机制保障时间轮状态与有效心跳任务保持一致，避免失效 Agent 引发误唤醒与无效扫描，维持调度正确性。

心跳属于周期性定时器，单次唤醒不代表任务终止。若 Agent 持续启用心跳周期，触发唤醒后调用 `agent_heartbeat_wheel_reschedule()`，重新计算下一次 `heartbeat_deadline`，复用插入逻辑将 Agent 重新注册至未来桶。时间轮依靠重排机制持续维护周期性任务调度位置，形成持续运转的动态调度结构。

3.5.3 统一事件置位逻辑

消息抵达、心跳到期、文件修改等所有唤醒触发源，均经由 `agent_signal_event_locked()` 统一处理。系统采用 `pending_events` 事件位图聚合所有待处理事件，执行逻辑：

- 将事件标记并入 `pending_events`；
- 记录唤醒原因与唤醒时刻；
- 适度提升 Agent 调度优先级；
- 若 Agent 处于阻塞等待状态，将其切换为可运行态。

该设计消除多唤醒源各自独立的唤醒分支，构建统一事件总线，调度器仅需查询 Agent 事件位图即可判定是否需要唤醒，简化内核调度逻辑。

● Agent 主循环等待入口

`agent_proc_wait()` 是 Agent 运行循环的核心阻塞接口，决定循环终止或进入事件等待，分为两种运行模式：

`continue_loop == 0`：终止 Agent 循环，清理等待上下文，退出执行流程；

`continue_loop != 0`：进入阻塞等待。等待阶段更新核心状态：

`loop_state = WAITING`：标记 Agent 阻塞；

`wait_generation++`：生成等待代次，区分新旧等待上下文；

写入 `wait_timeout_ticks`、`wait_deadline`，约束等待最长时长。

接口并非简单休眠，而是向系统登记一次带有超时边界、可区分代次的阻塞等待请求。

● 事件返回优先级规则

等待触发唤醒后，系统按照固定优先级返回事件，优先级从高至低：

PARENT_GONE > MESSAGE > FILEMOD > HEARTBEAT > TIMEOUT > CANCELLED

优先级体现系统调度策略：父进程消亡等生命周期治理事件优先级最高；其次为 Agent 间协作消息；文件环境变更次之；周期性心跳自检优先级低于外

部主动事件；超时、任务取消作为兜底返回条件。整体优先保障控制类事件与跨主体协作通信。

● 4.5 超时与无源等待保护机制

`agent_proc_wait()`内置两类防御性逻辑，规避异常阻塞：

`AGENT_WAIT_NO_SOURCE` 若 Agent 未订阅任何 `watch` 事件、无周期性心跳、消息队列为空且未配置等待超时，不存在任何潜在唤醒条件，直接返回该状态，防止 Agent 永久阻塞睡眠。

`AGENT_WAIT_TIMEOUT` 当等待时长抵达 `wait_deadline` 仍无事件触发，则返回超时。支持定时轮询、重试逻辑，让等待机制同时兼容事件驱动与周期性主动检查。

3.5.4 软预算机制

当前实现强调的是 `soft budget`，而不是硬性禁止运行，预算耗尽不会直接阻止调度，只是降低得分。这让系统既能控制资源倾斜，又不至于饿死关键 Agent。

Agent 的调度优势主要来自 `agent_schedule_score()`它对 `RUNNABLE` 进程计算 `score`。若 `agent_sched_budget <= 0`，则扣减 `AGENT_BUDGET_PENALTY_SCORE`，并增加 `agent_sched_budget_penalty`。

`Score` 的主要来源包括：

- 基础优先级
- `aging`
- `pending event` 权重
- `agent_sched_boost`
- `vruntime` 惩罚
- `budget` 用尽惩罚

即将超时 `bonus`

同时，`agent_event_weight()` 定义了不同事件的加权：

- `MESSAGE` 最高
- `PARENT_GONE` 次高
- `FILEMOD` 再次
- `HEARTBEAT` 相对较低

`agent_replenish_budgets()` 会周期性遍历 Agent。若到达 `budget_replenish_deadline`，且有配置 `agent_sched_quota`，则把 `agent_sched_budget` 重置为 `quota`。该函数在 `agent_tick()` 中周期调用。

3.6 安全控制与多 Agent 协作治理

安全控制模块在先前几个模块的实现中，有所解释阐述，在此章节进行总结性展示。

3.6.1 角色与 capability 模型

● capability 位图

系统中定义了完整 capability 位图：

- ✧ AGENT_CAP_QUERY_PROCESS
- ✧ AGENT_CAP_QUERY_FILE
- ✧ AGENT_CAP_READ_FILE
- ✧ AGENT_CAP_PATCH_FILE
- ✧ AGENT_CAP_SEND_MESSAGE
- ✧ AGENT_CAP_WATCH_FILE
- ✧ AGENT_CAP_REGISTER_TOOL
- ✧ AGENT_CAP_SCHED_CONFIG
- ✧ AGENT_CAP_LEASE_ACQUIRE
- ✧ AGENT_CAP_WORKFLOW_CTRL
- ✧ AGENT_CAP_AUDIT_READ
- ✧ AGENT_CAP_ALL

agent_role_default_caps() 定义了角色默认能力，例如：

- ✧ Planner
进程查询、文件查询、消息发送、工作流控制、调度配置、审计读取
- ✧ Retriever
文件查询、读文件、消息发送
- ✧ Patch
读文件、改文件、申请租约、消息发送
- ✧ Reviewer
读文件、监听文件、消息发送、文件查询
- ✧ Tool-Service
注册工具、消息发送

一旦设置角色，自动赋予默认 capability 并锁定角色。

agent_proc_cap_set() 的关键约束更严格，只能在已有能力集合内“削减”，不能

凭空增加新权限。若尝试权限提升，则拒绝并写审计。运行期只允许降权，不允许自提升。

3.6.2 工具权限控制

- **内建工具权限**

`builtin_policies[]` 为每个内建工具指定所需 `capability`。例如：

`patch_file` 需要 `AGENT_CAP_PATCH_FILE`

`query_file` 需要 `AGENT_CAP_QUERY_FILE`

`send_message` 需要 `AGENT_CAP_SEND_MESSAGE`

`lease_begin/commit/abort` 需要 `AGENT_CAP_LEASE_ACQUIRE`

- **动态工具权限与可见性**

动态工具除了可能要求 `capability` 外，还要满足同组调用，或工具被标记为 `PUBLIC` 两个条件。

这避免了某个 Tool-Service Agent 随意把内部工具暴露给所有 Agent。

3.6.3 Context Guard Page 与校验设计

- **Guard Page**

`agent_mark_current()` 在分配 Context 区时，会额外分配一页 `AGENT_GUARD_PAGE_SIZE`，并通过 `uvmclear()` 清除用户访问权限。

Context 区尾部后一页保留在地址空间中，但用户态不可访问，若发生越界写读，可更早暴露问题。这是一种典型内存安全保护策略。

- **generation 奇偶一致性**

`agent_context_write_begin()` 与 `agent_sync_header()` 共同实现了 generation 奇偶协议：

奇数 generation 表示正在写入；

偶数 generation 表示稳定快照。

这防止了用户态读取到半写 Header、数据区和头部不一致的问题。

- **digest 链式校验**

`agent_context_digest_push()` 会为每条 Context 记录构造链式摘要。

`agent_context_digest_verify()` 会检查以下内容，以保证历史链的完整性：

- ✧ `head/count` 是否合法
- ✧ `previous_hash` 链是否连续
- ✧ `sequence` 是否严格单调
- ✧ `current_hash` 是否可重算一致

3.6.4 Workflow 与级联终止治理

● workflow 标识

在 struct proc 中， workflow 治理相关字段包括：

- ✧ workflow_id
- ✧ workflow_leader_pid
- ✧ workflow_leader_generation
- ✧ agent_parent_pid
- ✧ agent_parent_generation
- ✧ is_workflow_leader
- ✧ workflow_member_registered

在全局 agent_global 中还维护了 workflow 表。

agent_cascade_terminate() 是协作治理的关键函数。它要求：

- ✧ 调用者是 Agent；
- ✧ 要么是 workflow leader；
- ✧ 要么具备 AGENT_CAP_WORKFLOW_CTRL；
- ✧ 当前必须隶属于合法工作流。

执行时它会：

- ✧ 将 workflow 状态切到 TERMINATING
- ✧ 遍历所有成员
- ✧ 停止心跳
- ✧ 清空 watch
- ✧ 注入 AGENT_EVENT_PARENT_GONE
- ✧ 若成员在睡眠则唤醒

这使 workflow 治理具备一致性终止语义，而不是依赖用户态自行通知。

● fork 与继承治理

agent_after_fork() 中还体现了一个重要安全原则，子进程继承 Agent 身份类型和组信息，但不会继承 capability，角色也重置为未设置。需要重新认证和重新赋权。

这是一种继承身份，不继承权限的保守策略。

4 CodeLab 综合场景

4.1 场景架构

本项目的多 Agent 代码修复演示场景，灵感源自 Claude Code 等主流 AI 编程智能体的工作范式，并在 xv6 操作系统内核中完成了机制层面的模拟复现。现实中 AI 编程 Agent 依靠循环工具调用、文件检索、代码修改、自动化测试等流程完成代码缺陷修复。

本场景提取该 workflow 核心逻辑，搭建了一套运行于操作系统内核的仿真闭环：预置存在计数缺陷的 todo 代码仓库，拆分规划、检索、修改、测试、审核、工具服务六类专用智能体，还原 AI 编程“查找代码漏洞 — 自动打补丁 — 校验修复效果 — 审核变更”的完整链路，同时针对原生 AI 编程工具存在的重复文件查询、工具能力固化、进程调度无优先级区分等痛点，新增跨 Agent 共享查询缓存、动态工具注册、事件感知调度三大内核创新机制，将原本运行在应用层的 AI 智能体逻辑下沉至操作系统。

CodeLab 场景预置一个包含 bug 的小型代码仓库，使用 6 个 Agent 协作完成修复：

角色	程序文件	职责
Planner-Agent	planner_agent.c	总控：创建 worker，分发任务，收集结果
Retriever-Agent	retriever_agent.c	检索：query_file + read_file 定位 bug
Patch-Agent	patch_agent.c	修复：lease_begin → patch_file → lease_commit
Test-Agent	test_agent.c	测试：调用动态工具 run_rule_test_dyn
Reviewer-Agent	reviewer_agent.c	审核：监听 FILEMOD, diff_file, approve/reject
Tool-Service	rule_test_tool_agent.c	服务：注册动态工具，执行规则测试

4.2 对比测试

为验证 AgentOS-AAA 在多 Agent 协作代码修复场景中的有效性，本文选取 plain xv6 与 AgentOS 两条执行路径进行对照实验。两条路径承担完全相同的业务任务，即定位 repo/todo.c 中 delete_task 函数的计数缺陷，插入 task_count--；修复语句，执行规则测试，并完成最终审核确认。

在文件检索与访问成本方面，AgentOS 显示出显著优势。

实验结果表明，plain xv6 共扫描 122 个文件对象，而 AgentOS 仅扫描 4 个，扫描量下降 96.72%，前者约为后者的 30.50 倍。该结果说明，AgentOS 中 AgentFS 的属性索引与候选集筛选机制有效地将传统“全量遍历后再筛选”的检索模式转化为“先基于索引缩小候选范围，再进行精确匹配”的检索模式。与此相一致，文件读取次数由 125 次下降至 2 次，降幅达到 98.40%；文件读取字节数由 22319 字节下降至 1022 字节，降幅为 95.42%，plain xv6 的读取数据量约为 AgentOS 的 21.84 倍。由此可见，AgentOS 在文件定位阶段显著减少了对无关文件内容的反复打开与读取，体现出基于语义检索与结果缓存的明显优势。

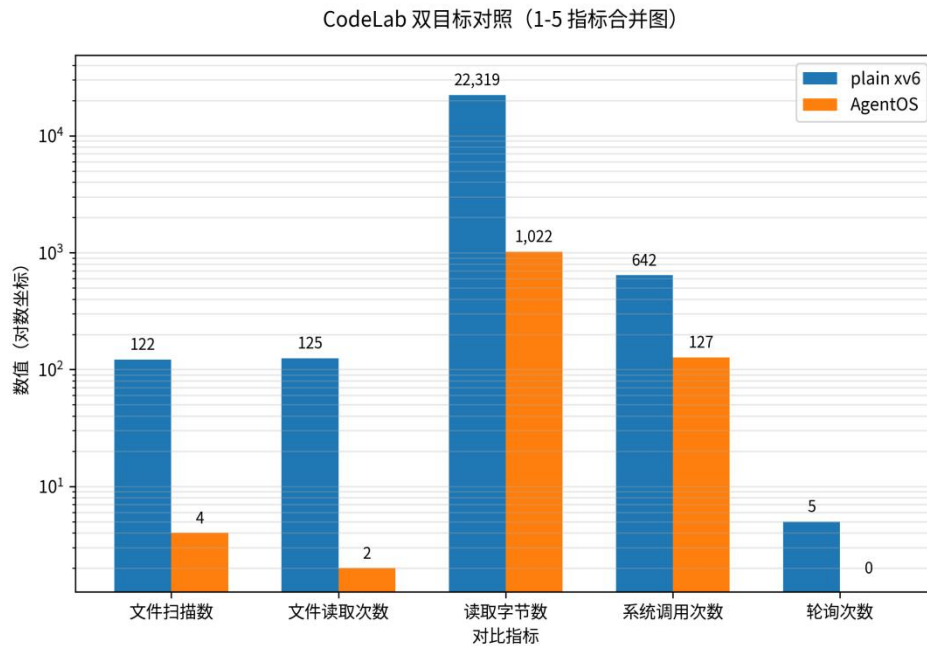


图 1.1 文件检索对比

在系统调用与等待方式方面，AgentOS 同样表现出较高的执行组织效率。实验中，plain xv6 共发生 642 次系统调用，而 AgentOS 为 127 次，系统调用总数下降 80.22%。此外，plain xv6 在用户态发生了 5 次轮询，而 AgentOS 轮询次数为 0。这一差异表明，plain xv6 路径仍较多依赖“主动反复检查状态”的工作方式，而 AgentOS 通过 `agent_wait` 以及 `MESSAGE`、`FILEMOD` 等事件机制，将等待过程转化为内核统一事件驱动模型，从而消除了用户态轮询开销。换言之，AgentOS 在执行模式上更接近一种“按事件唤醒、按协作推进”的组织方式，而非简单轮询式流程控制。

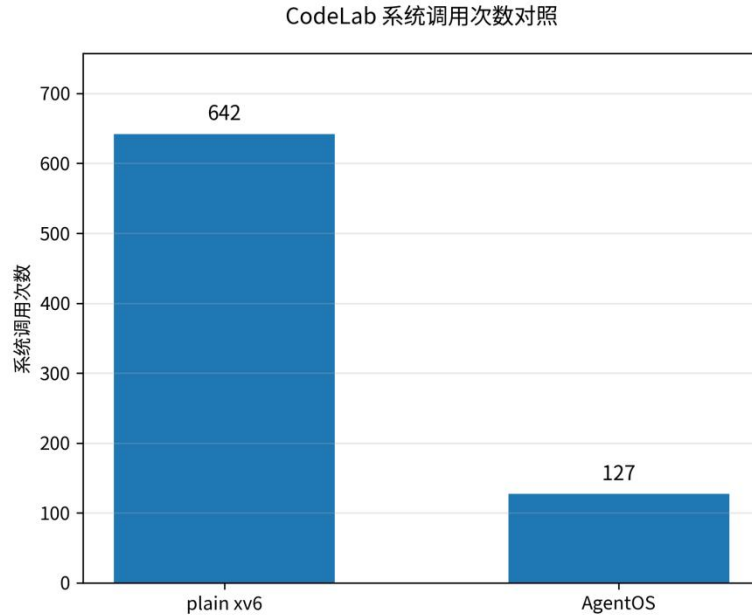


图 4.2 系统调用对比

然而，在总耗时指标上，当前实验并未显示 AgentOS 在小规模任务上的绝对时间优势。实验结果表明，plain xv6 总耗时为 8 ticks，而 AgentOS 为 44 ticks。

因此，不能简单将 AgentOS 的实验结果概括为整体执行更快。在当前小型 CodeLab 任务中，AgentOS 为多 Agent 创建、消息传递、Context 记录、动态工具分发、事件等待以及审核闭环等系统机制支付了较高的固定管理开销。这些开销在任务规模较小时占比突出，从而掩盖了其在文件访问和查询路径上的优化收益。

4.3 测试结果

AgentOS 在文件扫描量、文件读取次数、读取字节数、系统调用总数以及用户态轮询次数等资源访问指标上均显著优于 plain xv6，证明其 AgentFS、事件等待与共享缓存机制能够有效减少无关访问和重复查询。

再次，AgentOS 当前在总耗时上仍高于 plain xv6，反映出其为完整多 Agent 协作、Context 管理、动态工具和执行可解释性所支付的额外系统成本。AgentOS-AAA 在当前小规模任务上尚未表现出整体执行时间优势，但已经显示出显著的文件访问优化与协作机制优势。随着仓库规模扩大、文件数量增加以及任务链条变长，这种减少无关访问、提升协作组织能力的优势有望进一步累积，并可能逐步转化为更明显的总体性能收益。

5 实际 LLM 测试

本项目采用虚拟机桥接层 + 宿主机代理分层测试架构：

xv6 侧：

planner_agent 主控 Agent、llm_bridge 桥接程序，负责封装标准化 LLM 请求、接收模型返回动作指令，仅做消息收发，无推理逻辑；

宿主机侧：

llm_proxy.py、llm_qemu_driver.py 两套测试脚本，提供离线模拟、真实 API 两种测试模式：

测试核心逻辑：LLM 仅输出高层任务决策（是否启动代码修复流水线），文件检索、修改、测试、审核等底层执行逻辑全部由 AgentOS 内核与多 Agent 完成，实现“LLM 做策略，操作系统做执行”的分层验证目标。

```

$ tanyx_dawn@Tanyx:~/gitStore/project3136859-388760$ python3 tools/llm_qemu_driver.py --mode api
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio
-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus
=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ planner_agent llm-api
[Planner-Agent] task: fix todo delete bug
[Planner-Agent] waiting for first HEARTBEAT
[Kernel-AgentLoop] HEARTBEAT -> wakeup Planner-Agent
[LLM-Bridge] api model selected; waiting for host proxy decision
@@AGENTOS_LLM_REQ id=planner role=planner prompt=fix todo delete bug @@END
[LLM-Bridge] waiting for host proxy response line

[Host-Proxy] send response: @@AGENTOS_LLM_RESP id=planner state=done text=ACTION_start_codelab_REASON_User_
requested_fix. action=start_codelab @@END
@@AGENTOS_LLM_RESP id=planner state=done text=ACTION_start_codelab_REASON_User_requested_fix. action=start_
codelab @@END
[LLM-Bridge] received host response: @@AGENTOS_LLM_RESP id=planner state=done text=ACTION_start_codelab_REA
SON_User_requested_fix. action=start_codelab @@END
[Planner-Agent] plan: find files -> inspect bug -> patch -> test -> review
[Planner-Agent] repo file read ok, bug marker not in first chunk
[Kernel] create Retriever-Agent pid=4
[Kernel] create Patch-Agent pid=5
[Kernel] create Test-Agent pid=6
[Kernel] create Reviewer-Agent pid=7
[Kernel] create Tool-Service pid=8
[Planner-Agent] spawned retriever=4 patch=5 test=6 reviewer=7 tool=8
[Scheduler] Planner-Agent sched=4/3 heartbeat=12
[Scheduler] Retriever-Agent high priority sched=8/8
[Scheduler] Patch-Agent high priority sched=8/7
[Scheduler] Test-Agent on-demand sched=5/4
[Scheduler] Reviewer-Agent filemod/message sched=7/6
[Scheduler] Tool-Service sched=4/3
[Tool-Service] register run_rule_test_dyn ok; stage=tool;status=ready;tool=run_rule_test_dyn;sched=4/3
[Patch-Agent] ready for MESSAGE wakeup; stage=patch;status=ready;sched=8/7
[Test-Agent] ready for patched-file message; stage=test;status=ready;sched=5/4
[Reviewer-Agent] watch FILEMOD repo/todo.c; stage=reviewer;status=watching;file=repo/todo.c;sched=7/6
[Planner-Agent] send_message Retriever-Agent: find delete_task related files
[Kernel-AgentLoop] MESSAGE -> wakeup Retriever-Agent
[Kernel-FS] query_file type=code,module=todo,keyword=delete -> cache_hit=0, used_index=1
[Retriever-Agent] indexed AgentFS query populated shared cache
[Kernel-FS] reviewer repeated query -> shared query cache hit
[Retriever-Agent] read file repo/todo.c -> found missing task count--

```

图 5.1 LLM 接入结果

6 性能测试

6.1 文件查询扫描量对比

- 测试目标：

验证 AgentFS 属性索引查询比 mode=scan 全表扫描检查更少候选文件。

- 测试结果

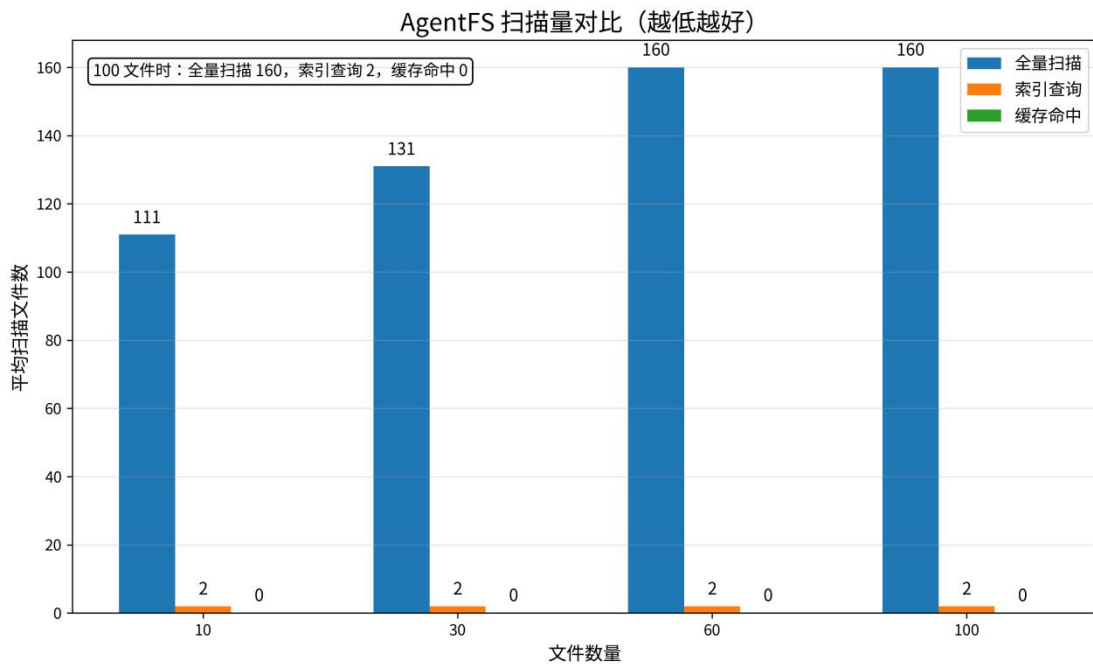


图 6.1 测试结果

在 AgentFS 文件检索方面，随着文件规模扩大，全量扫描路径的检查开销由 111 增长至 160，而索引查询路径始终仅需检查 2 个候选项，缓存命中时扫描量进一步降为 0。

这表明 AgentFS 已经有效地将传统文件系统中“大范围遍历后再筛选”的检索方式，转化为“先依赖属性索引缩小候选集合，再执行少量精确匹配”的检索模式。也就是说，系统的主要收益并不来自单次扫描速度的微小优化，而是来自检索空间本身的压缩。

该结果说明，AgentFS 的属性索引与语义化查询机制能够在文件规模增大时保持稳定的候选集规模，从而显著降低无关扫描成本。

6.2 共享缓存复用效果

● 测试目标：

验证 shared cache 复用是否有效，测试文件更改后 shared cache 机制的正确性。

● 测试结果：

从实验数据看，共享缓存机制在三个阶段表现出清晰的状态差异：首次查询需要执行真实扫描，扫描量为 1；当后续发起相同查询时，系统成功命中共享缓存，扫描量直接降为 0；而在文件属性发生变化后，原有缓存又会被自动判定失效，系统重新执行真实查询，扫描量恢复为 1。

首次查询需要执行真实扫描，缓存命中后扫描量可直接降为 0，而在文件属性发生变化后，系统又能够自动使旧缓存失效，并重新执行真实查询。这一现象说明共享缓存不仅能够有效复用重复查询结果，而且其失效机制也能够与文件状态变化保持一致。换言之，当前共享缓存并非单纯追求命中率，而是在结果复用与状态一致性之间建立了稳定平衡。

这表明 AgentOS 中的共享缓存已具备面向协作查询场景的实用价值，即在减少重复检索成本的同时，不会因状态变化而错误沿用陈旧结果。

6.3 AgentOS CPU 开销与唤醒延迟

● 测试目标：

AgentOS 的事件驱动机制把反复检查状态变成阻塞等待 + 内核唤醒，是否能够减少无效 CPU 消耗。

● 测试结果：

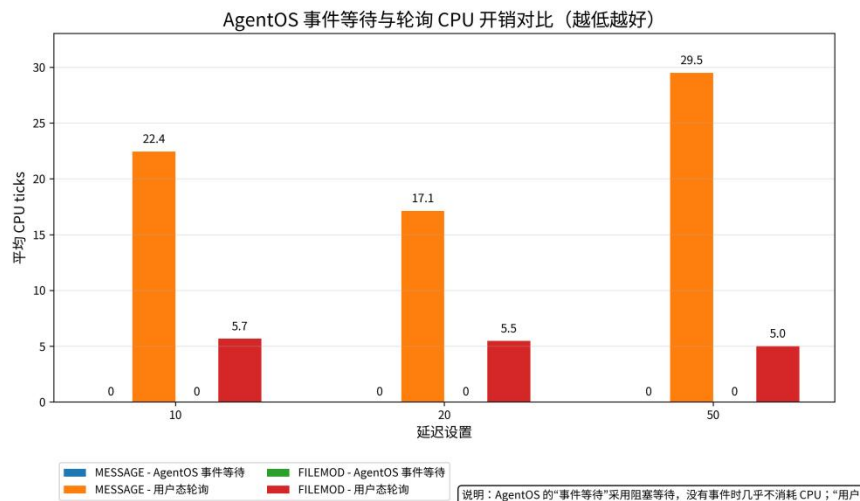


图 2.2 CPU 开销对比

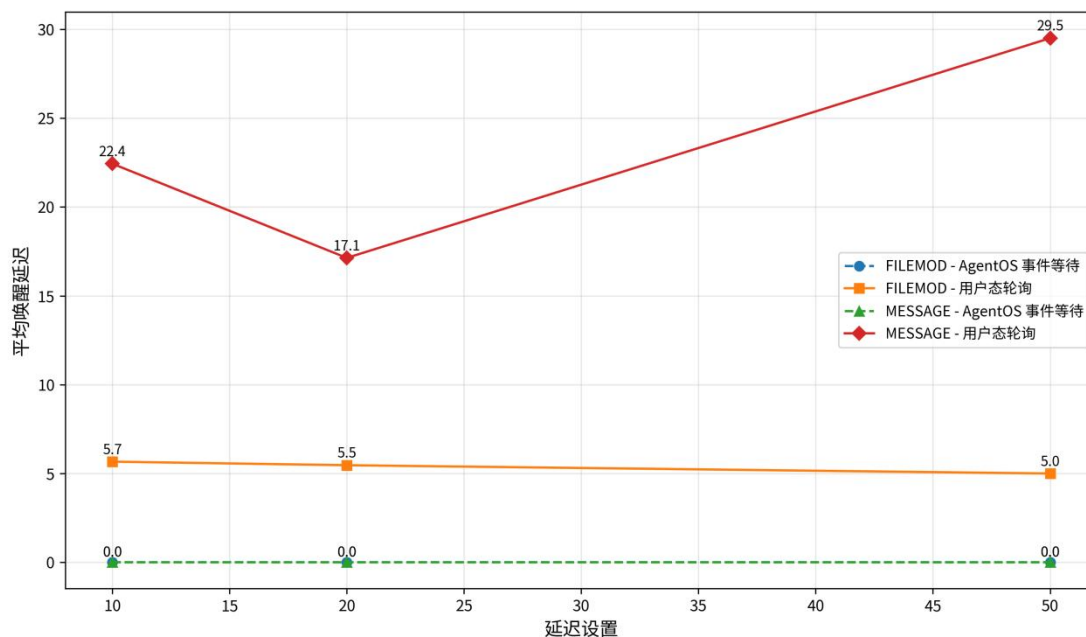


图 6.3 唤醒延迟对比

实验表明，对于 MESSAGE 与 FILEMOD 两类典型事件，agent_wait 在等待阶段的平均 CPU ticks 始终为 0，而用户态轮询方式仍持续消耗 CPU 时间，并且消耗量会随延迟变化而波动。与此同时，AgentOS 在唤醒延迟上并未表现出额外代价，MESSAGE 与 FILEMOD 事件的平均唤醒延迟均保持在 0 tick。该结果说明，AgentOS 已成功将传统的“反复检查状态”模式替换为“阻塞等待 + 内核唤醒”模式，从而显著降低等待阶段的无效 CPU 消耗，并在此基础上保持了较低的唤醒延迟。这证明事件驱动等待机制在当前实现中不仅成立，而且已经表现出明确的系统收益。

6.4 调度机制公平性

● 测试目标：

测试调度机制是否实现优先级管理，是否出现低优先级进程饿死。

● 测试结果

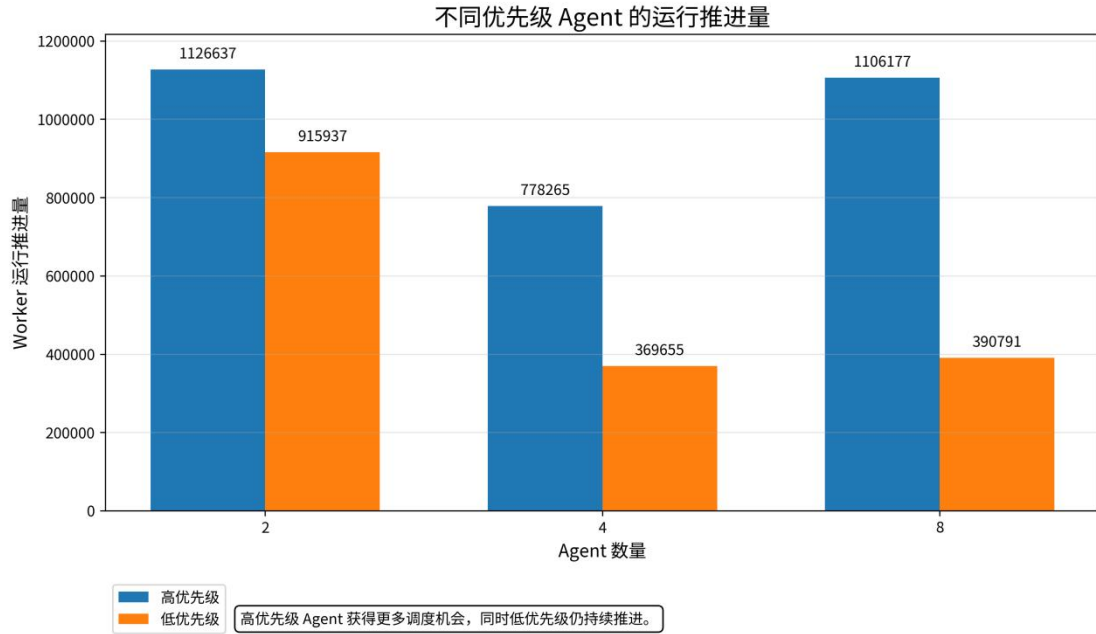


图 6.4 Agent 推进量

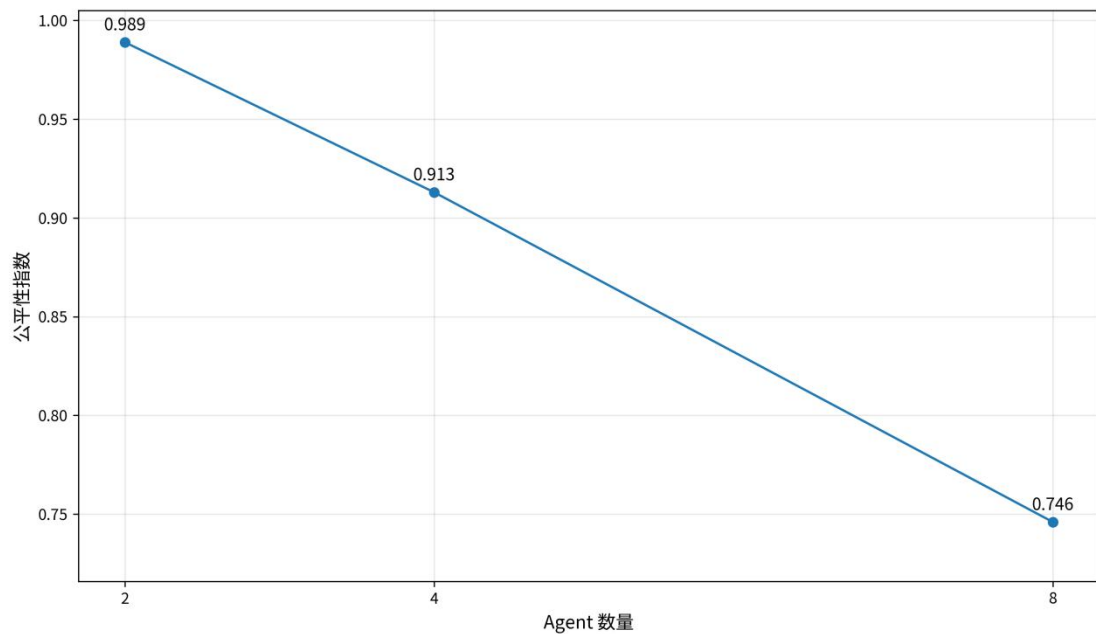


图 6.5 调度公平性

不同优先级的 Agent 在运行推进量上存在稳定差异：高优先级 Worker 在 2、4、8 个 Agent 规模下的推进量均显著高于低优先级 Worker，高低推进量之比由 1.23 倍逐步上升至 2.83 倍。这表明 AgentOS 的调度器确实能够根据优先级策略主动向高优先级任务倾斜资源，而不是停留在理论设计层面。同时，低优先级 Worker 的推进量始终大于 0，说明系统虽然存在优先级倾斜，但并未出现低优先级任务被完全饿死的现象。结合 Jain 公平性指数可以看到，随着 Agent 数量增加，公平性指数从 0.989 下降至 0.746，这表明调度器

在优先级响应与整体公平性之间进行了主动权衡。因此，当前实验更适合说明“调度器能够在保持低优先级任务持续推进的同时，对高优先级任务实施资源倾斜”，而不应简单表述为“系统规模越大越公平”。

7 AI 使用情况

本项目在开发阶段引入业界主流 AI 编程工具 **Codex** 辅助全流程开发，核心应用覆盖**代码阅读与模块定位、局部功能实现、缺陷定位与程序调试、技术文档与资料整理**等多个环节。项目初期，借助 Codex 对 xv6 原始内核代码、Agent 扩展模块以及用户态程序进行结构化阅读，快速梳理 ``proc``、``agent_context``、``agent_fs``、``agent_loop``、``agent_tool`` 等核心模块之间的调用关系与数据流向，用于定位关键实现入口和理解系统整体架构。在局部实现阶段，Codex 主要用于辅助补全部分非核心但工程量较大的功能代码，例如接口包装、测试程序辅助逻辑、调试输出整理以及部分文档化注释内容，从而提高开发效率。

在完成 xv6 内核扩展、多类 Agent 用户态程序以及 LLM 桥接代理脚本的初步编写后，项目组进一步借助 Codex 开展分层调试工作：对于内核进程管理、Agent 上下文内存分配、共享查询缓存、事件调度器等底层模块，结合编译报错信息、QEMU 运行 panic 日志以及工具调用异常返回结果，由 Codex 辅助分析并定位内存越界、PCB 字段初始化遗漏、IPC 消息阻塞、缓存脏数据残留等隐蔽逻辑问题；对于多 Agent 协作场景代码，则围绕 ``todo.c`` 缺陷修复流水线的实际报错，辅助梳理 ``query_file``、动态工具调用、FILEMOD 事件唤醒链路中的时序错误与参数不匹配问题。同时，项目还利用 Codex 对结构化工具请求/响应接口以及 LLM 串口通信报文解析逻辑进行边界校验，快速发现并修正工具名不存在、参数类型非法等异常处理分支中的潜在漏洞。

此外，在项目总结与答辩准备阶段，AI 工具还被用于辅助整理技术文档、实验结果说明、模块设计分析与汇报材料草稿，以提升资料归纳效率和文字表达规范性。需要说明的是，Codex 在本项目中主要承担辅助分析、局部实现与调试支撑作用，所有关键设计方案、核心代码实现、实验结果确认及最终文档内容，均经过项目成员人工审阅、修改和编译测试验证后方予以采纳。



图 7.1 ai 记录

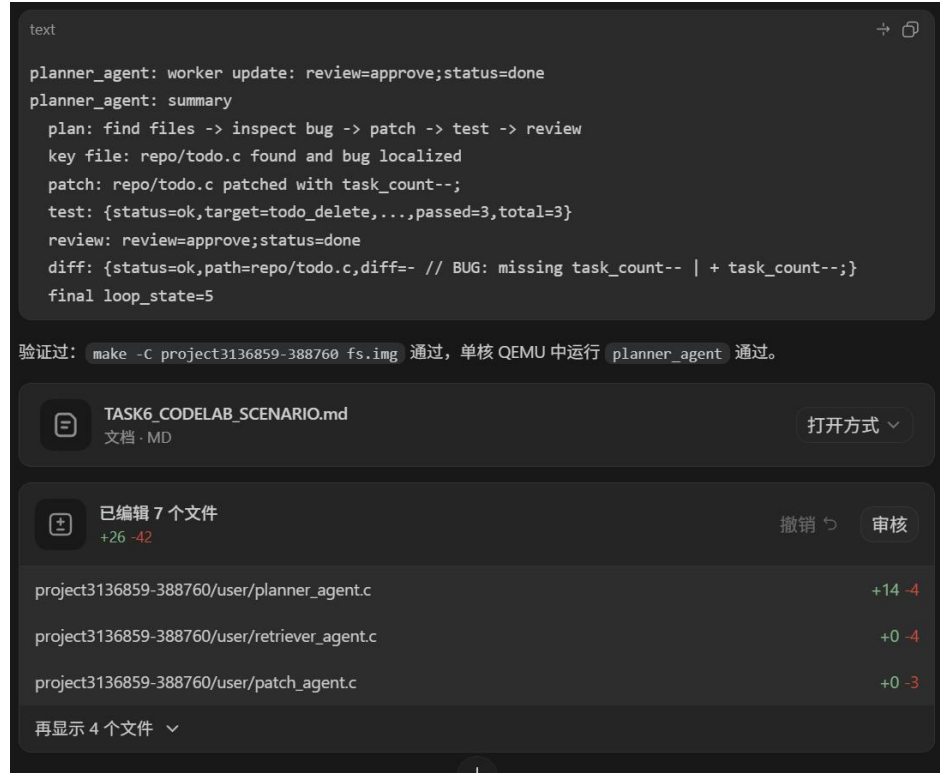


图 7.2 ai 记录

```
tanyx_dawn@Tanyx:~/gitStore/project3136859-388760$ python3 tools/llm_qemu_driver.py
--mode api
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -
nographic -global virtio-mmio.force-legacy=false -drive
file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-
bus.0

xv6 kernel is booting

init: starting sh
$ llm_bridge api fix_todo_delete
[LLM-Bridge] mode=api role=planner
@@AGENTOS_LLM_REQ id=1 role=planner prompt=fix_todo_delete @@END
[LLM-Bridge] waiting for host proxy response line

[Host-Proxy] send response: @@AGENTOS_LLM_RESP id=1 state=done
text=**Compact_Fix_Plan_-_Todo_Delete_Bug**_1_**Rep action=query_file @@END
@@AGENTOS_LLM_RESP id=1 state=done
text=**Compact_Fix_Plan_💎💎💎_Todo_Delete_Bug**_1_**Rep action=query_file @@END
[LLM-Bridge] received host response: @@AGENTOS_LLM_RESP id=1 state=done ...

显示更多 ▾
```

这是真实 API 已经调用成功了，但模型返回的文本里有非 ASCII 字符，例如 💎，经过 xv6 串口/console 显示时被替换成了乱码：

```
text
Compact_Fix_Plan_💎💎💎_Todo_Delete_Bug
```

关键点在这里：

```
text
[Host-Proxy] send response: @@AGENTOS_LLM_RESP id=1 state=done ...
[LLM-Bridge] received host response: @@AGENTOS_↓_RESP id=1 state=done ...
```

图 7.3 ai 记录

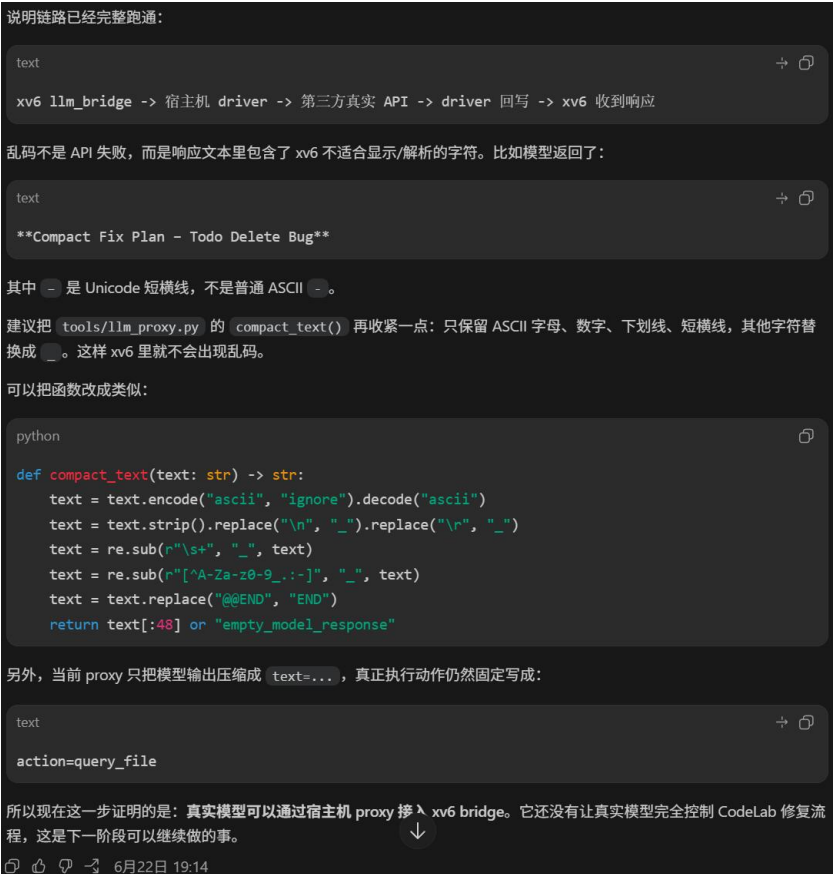


图 7.4 ai 记录



图 7.5 ai 记录



图 7.6 ai 记录